

合肥工業大學

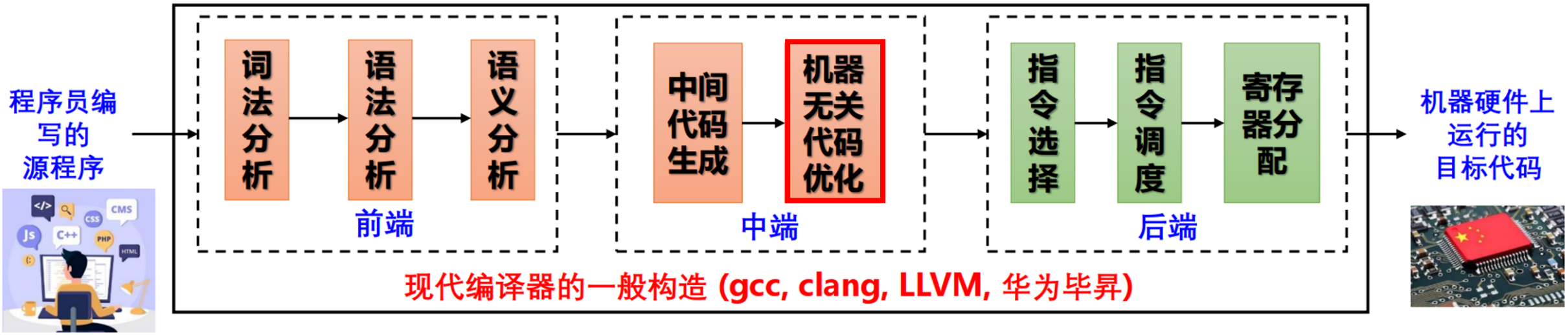


计算机与信息学院
SCHOOL OF COMPUTER SCIENCE AND INFORMATION ENGINEERING
人工智能学院
SCHOOL OF ARTIFICIAL INTELLIGENCE

机器无关代码优化
Part1：常见的优化方式

刘彬彬
计算机科学与技术系
2025年4月21日

本节提纲



- **代码优化的定义及背景**
- **常见的优化方法**
  - 公共子表达式删除
  - 死代码删除、复制传播、常量合并
  - 循环系列优化

什么是代码优化?

- 在不改变程序运行效果的前提下，对程序代码进行等价变换，使之能生成更加高效的目标代码。

- 优化目标
  - 运行时间更短
  - 占用空间更小

- 代码优化是编译程序过程中不可获取的环节!

为什么需要代码优化？

源头1：程序员编写的代码存在低效计算。

源头2：代码翻译过程中产生了冗余代码。



- **高级语言**
  - 直接面向开发者
  - 编程效率高
  - 与数学公式相似
- **编译器**
  - 实现人机交流和翻译
  - 目标代码可能效率低下
- **机器语言**
  - 驱动硬件完成具体任务
  - 编程效率低

举例——快速排序

- **最经典、效率最高的排序算法之一**

```
int partition(int a[ ], int m, int n) {
    int i = m - 1; // i指向左分区的最后一个元素
    int j = n;     // j指向右分区的最后一个元素
    int v = a[n];  // 选择最后一个元素作为枢轴

    while (1) {
        // 增加i直到找到一个不小于v的元素
        do i = i + 1; while(a[i] < v);
        // 减小j直到找到一个不大于v的元素
        do j = j - 1; while(a[j] > v);

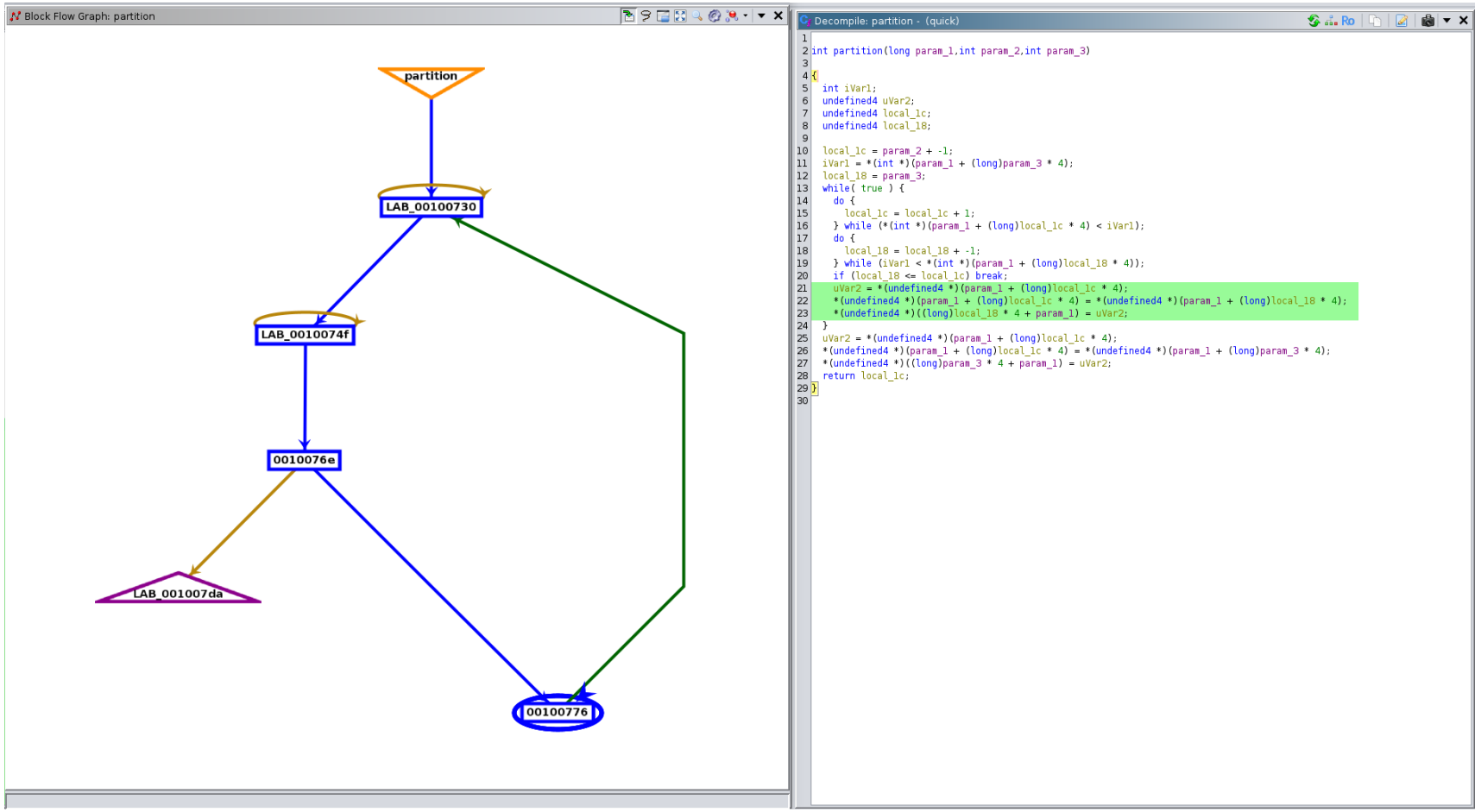
        // 如果i和j交叉，分区结束
        if (i >= j) break;
        // 交换i和j位置的元素
        int x = a[i];
        a[i] = a[j];
        a[j] = x;
    }

    // 最后交换枢轴到正确位置
    int x = a[i];
    a[i] = a[n];
    a[n] = x;

    return i; // 返回枢轴的最终位置
}
```

举例——快速排序

- gcc quicksort.c -o a.out
- 将a.out加载入工具Ghidra进行反编译



基本块流图与对应的反编译代码

举例——快速排序

- gcc quicksort.c -o a.out
- 将a.out加载入工具Ghidra进行反编译

Decompile: partition - (quick)

```
1
2  int partition(long param_1,int param_2,int param_3)
3
4  {
5    int iVar1;
6    undefined4 uVar2;
7    undefined4 local_1c;
8    undefined4 local_18;
9
10   local_1c = param_2 + -1;
11   iVar1 = *(int *)(param_1 + (long)param_3 * 4);
12   local_18 = param_3;
13   while( true ) {
14     do {
15       local_1c = local_1c + 1;
16     } while (*(int *)(param_1 + (long)local_1c * 4) < iVar1);
17     do {
18       local_18 = local_18 + -1;
19     } while (iVar1 < *(int *)(param_1 + (long)local_18 * 4));
20     if (local_18 <= local_1c) break;
21     uVar2 = *(undefined4 *)(param_1 + (long)local_1c * 4);
22     *(undefined4 *)(param_1 + (long)local_1c * 4) = *(undefined4 *)(param_1 + (long)local_18 * 4);
23     *(undefined4 *)((long)local_18 * 4 + param_1) = uVar2;
24   }
25   uVar2 = *(undefined4 *)(param_1 + (long)local_1c * 4);
26   *(undefined4 *)(param_1 + (long)local_1c * 4) = *(undefined4 *)(param_1 + (long)param_3 * 4);
27   *(undefined4 *)((long)param_3 * 4 + param_1) = uVar2;
28   return local_1c;
29 }
30
```

举例——快速排序

- gcc quicksort.c -o a.out
- 将a.out加载入工具Ghidra进行反编译

Decompile: partition - (quick)

```
1
2  int partition(long param_1,int param_2,int param_3)
3
4  {
5    int iVar1;
6    undefined4 uVar2;
7    undefined4 local_1c;
8    undefined4 local_18;
9
10   local_1c = param_2 + -1;
11   iVar1 = *(int *)(param_1 + (long)param_3 * 4);
12   local_18 = param_3;
13   while( true ) {
14     do {
15       local_1c = local_1c + 1;
16     } while (*(int *)(param_1 + (long)local_1c * 4) < iVar1);
17     do {
18       local_18 = local_18 + -1;
19     } while (iVar1 < *(int *)(param_1 + (long)local_18 * 4));
20     if (local_18 <= local_1c) break;
21     uVar2 = *(undefined4 *)(param_1 + (long)local_1c * 4);
22     *(undefined4 *)(param_1 + (long)local_1c * 4) = *(undefined4 *)(param_1 + (long)local_18 * 4);
23     *(undefined4 *)((long)local_18 * 4 + param_1) = uVar2;
24   }
25   uVar2 = *(undefined4 *)(param_1 + (long)local_1c * 4);
26   *(undefined4 *)(param_1 + (long)local_1c * 4) = *(undefined4 *)(param_1 + (long)param_3 * 4);
27   *(undefined4 *)((long)param_3 * 4 + param_1) = uVar2;
28   return local_1c;
29 }
30
```

编译器的优化选项







优化等级	简要说明
-Ofast	在-O3级别的基础上，开启更多激进优化项，该优化等级不会严格遵循语言标准
-O3	在-O2级别的基础上，开启更多的高级优化项。
-Os	在-O2级别的基础上，开启降低生成代码体量的优化
-O2	开启多数中级优化，会改善编译实践开销和最终生成代码性能
-O/-O1	优化效果介于-O0和-O2之间
-O0	不开启编译优化，只尝试减少编译时间

延伸阅读：https://clang.llvm.org/docs/CommandGuide/clang.html#code-generation-options

案例演示

- **1000000000次循环迭代累加**

```
void main() {
    int loop = 1000000000; // 循环次数定义
    long sum = 0;          // 累加变量初始化
    int start_time = clock(); // 开始计时
    int index = 0;

    // 循环体，执行累加操作
    for (index = 0; index < loop; index++) {
        sum += index;
    }

    int end_time = clock(); // 结束计时

    // 输出结果：累加和及代码运行时间
    printf("Sum : %ld, Time Cost : %lf \n", sum, (end_time - start_time) * 1.0 / CLOCKS_PER_SEC);
}
```

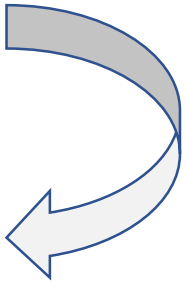
案例演示

- gcc –O0 无优化执行

```
robbert@robbert-virtual-machine:~/workspace/temp$ gcc -O0 performance-sample.c
robbert@robbert-virtual-machine:~/workspace/temp$ ./a.out
Sum : 4999999995000000000, Time Cost : 2.152898
```

- gcc –O1 中级优化执行

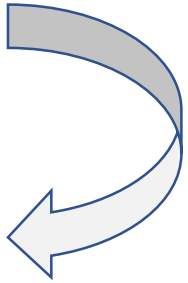
```
robbert@robbert-virtual-machine:~/workspace/temp$ gcc -O1 performance-sample.c
robbert@robbert-virtual-machine:~/workspace/temp$ ./a.out
Sum : 4999999995000000000, Time Cost : 0.252597
```



性能提升8倍

- gcc –O2 高级优化执行

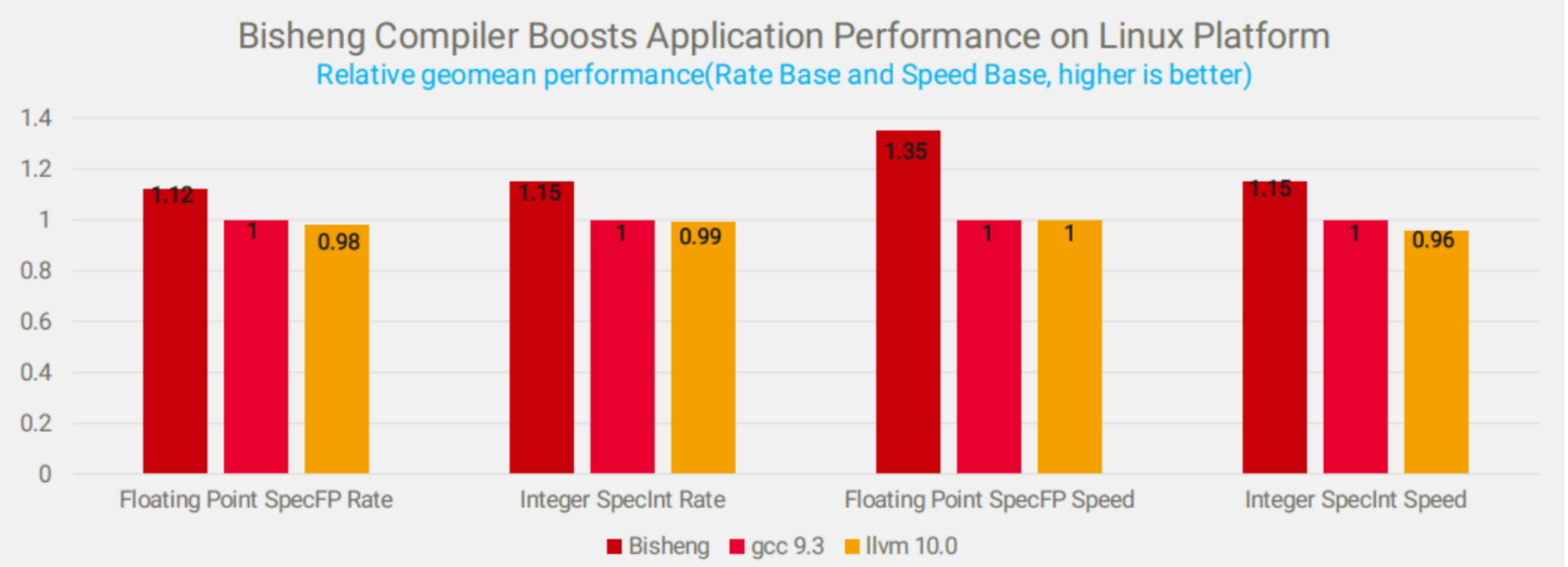
```
robbert@robbert-virtual-machine:~/workspace/temp$ gcc -O2 performance-sample.c
robbert@robbert-virtual-machine:~/workspace/temp$ ./a.out
Sum : 4999999995000000000, Time Cost : 0.000000
```



性能提升数十万倍

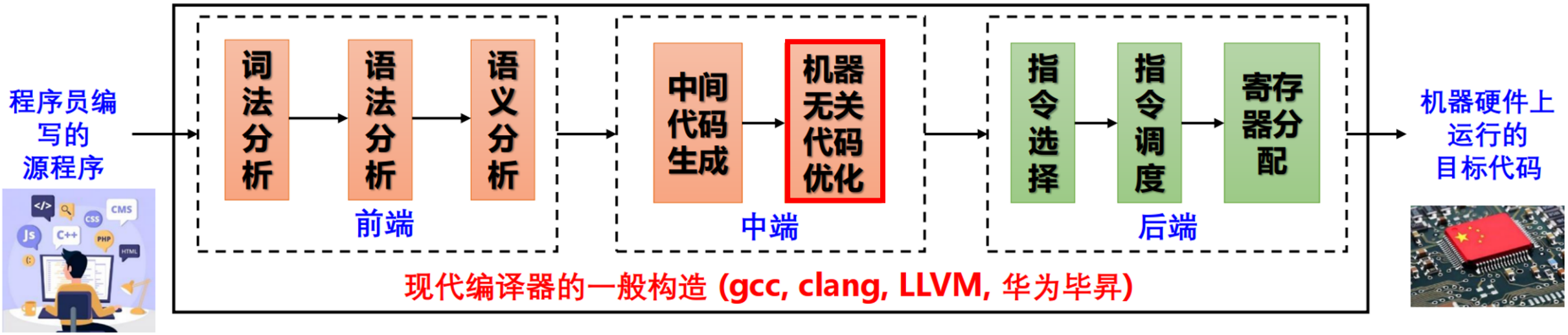
国产开源编译器——毕昇

- 基准测试集SPEC2017性能较业界平均高出15%以上



SPEC作为业界芯片性能评分标准，SPEC的分数可以直观体现出硬件的性能，越高越好。

本节提纲



- 代码优化的定义及背景
- **常见的优化方法**
  - 公共子表达式删除
  - 死代码删除、复制传播、常量合并
  - 循环系列优化

公共子表达式删除

- 公共子表达式
  - 如x op y已被计算过，且到现在为止，x和y的值未变，那么该表达式的本次出现是公共子表达式。

- 公共子表达式的删除
  - 本次计算可以被删除
  - 其值用上一次计算值代替

```
20      if (local_18 <= local_1c) break;
21      uVar2 = *(undefined4 *)(param_1 + (long)local_1c * 4);
22      *(undefined4 *)(param_1 + (long)local_1c * 4) = *(undefined4 *)(param_1 + (long)local_18 * 4);
23      *(undefined4 *)((long)local_18 * 4 + param_1) = uVar2;
24  }
```

举例——快速排序

- 对反编译程序进行变量分析和替换

```
int partition(long a, int m, int n) {
    int iVar1;          // 临时变量，用于存储值
    undefined4 uVar2;   // 临时变量，可能用于交换
    undefined4 i; // 分区的左指针
    undefined4 j; // 分区的右指针

    // 初始化分区指针和边界
    i = m + -1;             // 将 i 初始化为范围开始位置的前一个位置
    iVar1 = *(int *)(a + (long)n * 4); // 使用范围内的最后一个元素作为基准值（pivot）
    j = n;                  // 将右指针初始化为范围的末尾

    while (true) {                      // 开始分区逻辑的循环
        do {
            i = i + 1;    // 左指针向右移动
        } while (*(int *)(a + (long)i * 4) < iVar1); // 从左找到第一个大于等于基准值的元素
```

...

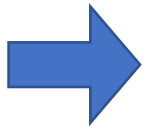
举例——快速排序

- 对反编译程序进行变量分析和替换
- 代码简化

```
int partition(long a, int m, int n) {
    int iVar1;          // 临时变量，用于存储值
    undefined4 uVar2;   // 临时变量，可能用于交换
    undefined4 i; // 分区的左指针
    undefined4 j; // 分区的右指针

    // 初始化分区指针和边界
    i = m + -1;             // 将 i 初始化为范围开始位置的前一个位置
    iVar1 = *(int *)(a + (long)n * 4); // 使用范围内的最后一个元素作为基准值（pivot）
    j = n;                  // 将右指针初始化为范围的末尾

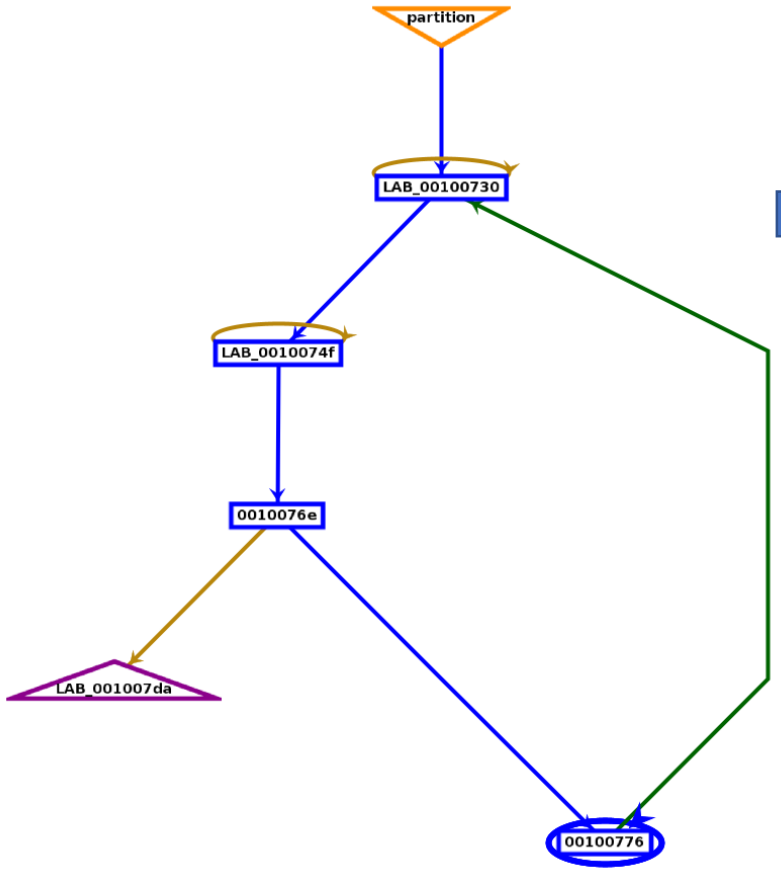
    while (true) {                  // 开始分区逻辑的循环
        do {
            i = i + 1;    // 左指针向右移动
        } while (*(int *)(a + (long)i * 4) < iVar1); // 从左找到第一个大于等于基准值的元素
```

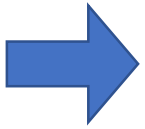


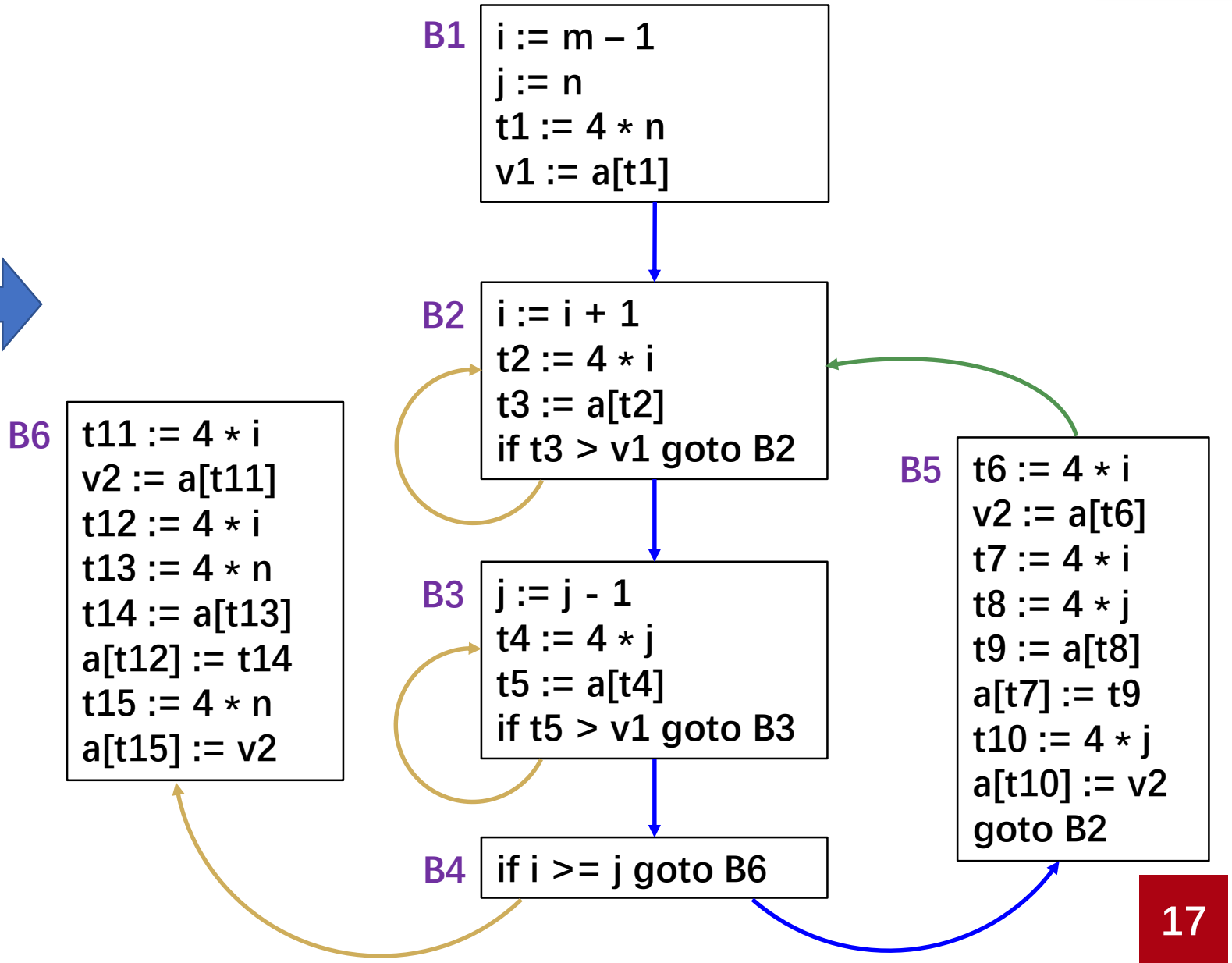
```
i := m – 1
j := n
t1 := 4 * n
v1 := a[t1]
```

...

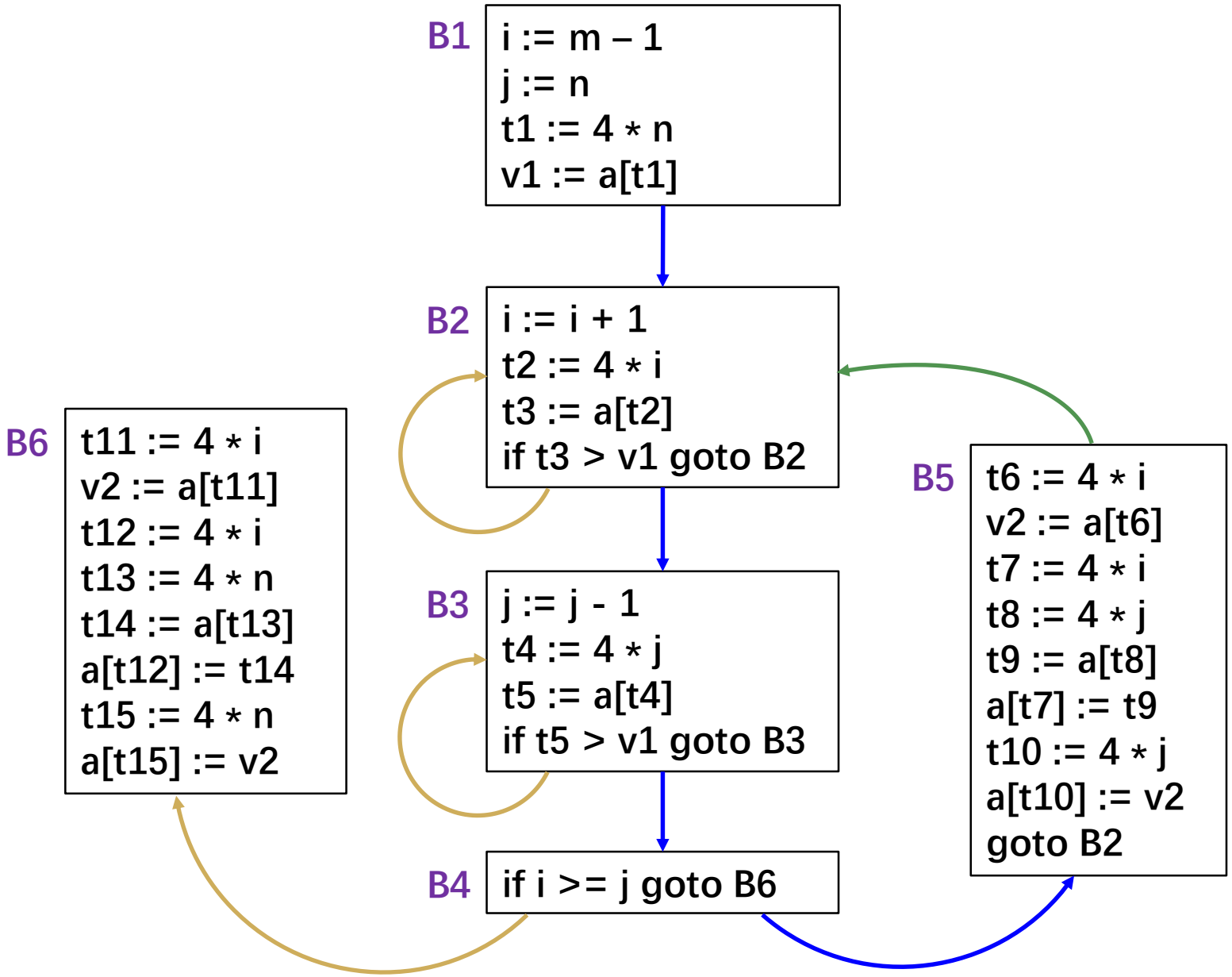
快排中的公共子表达式删除



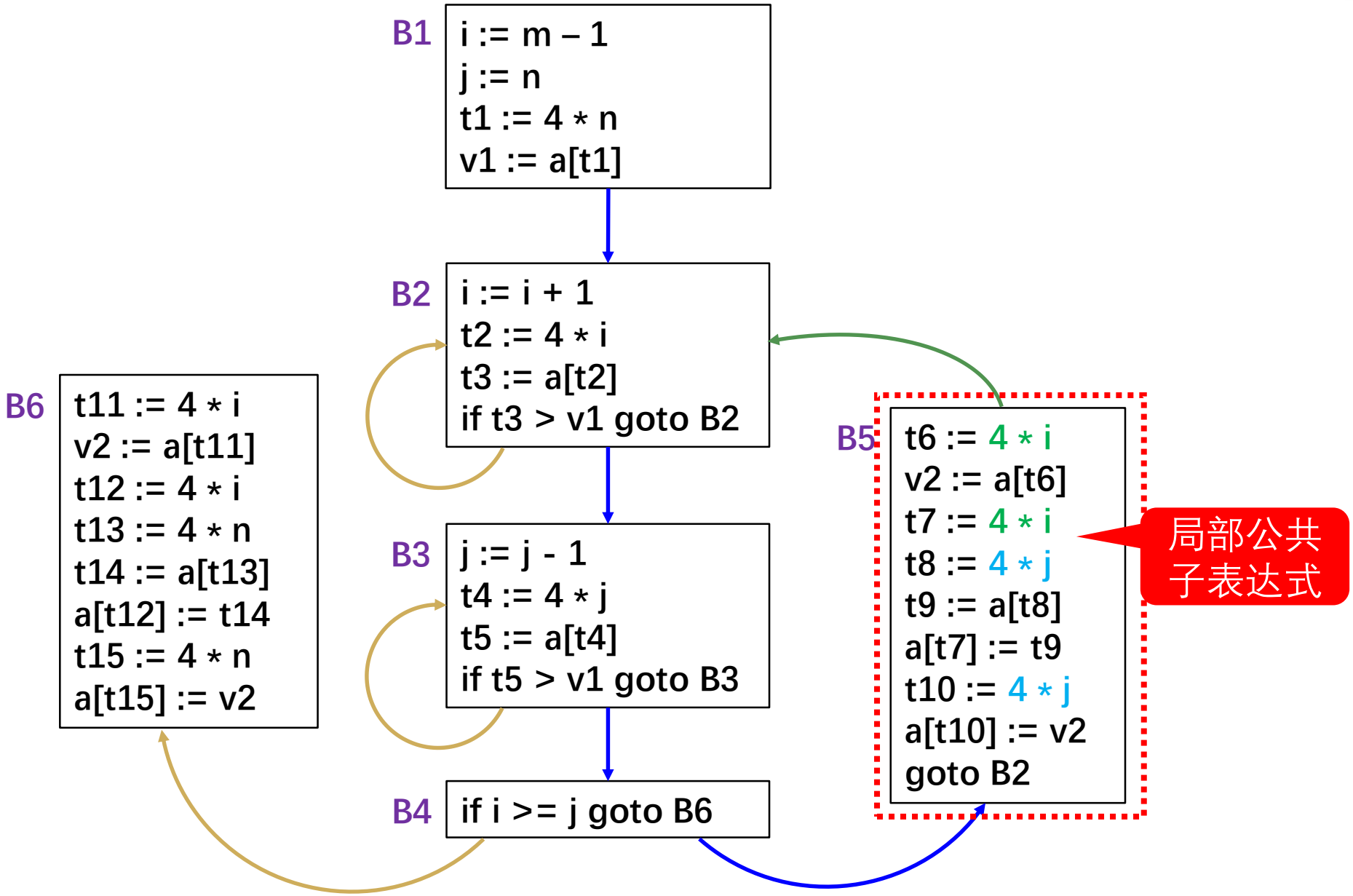




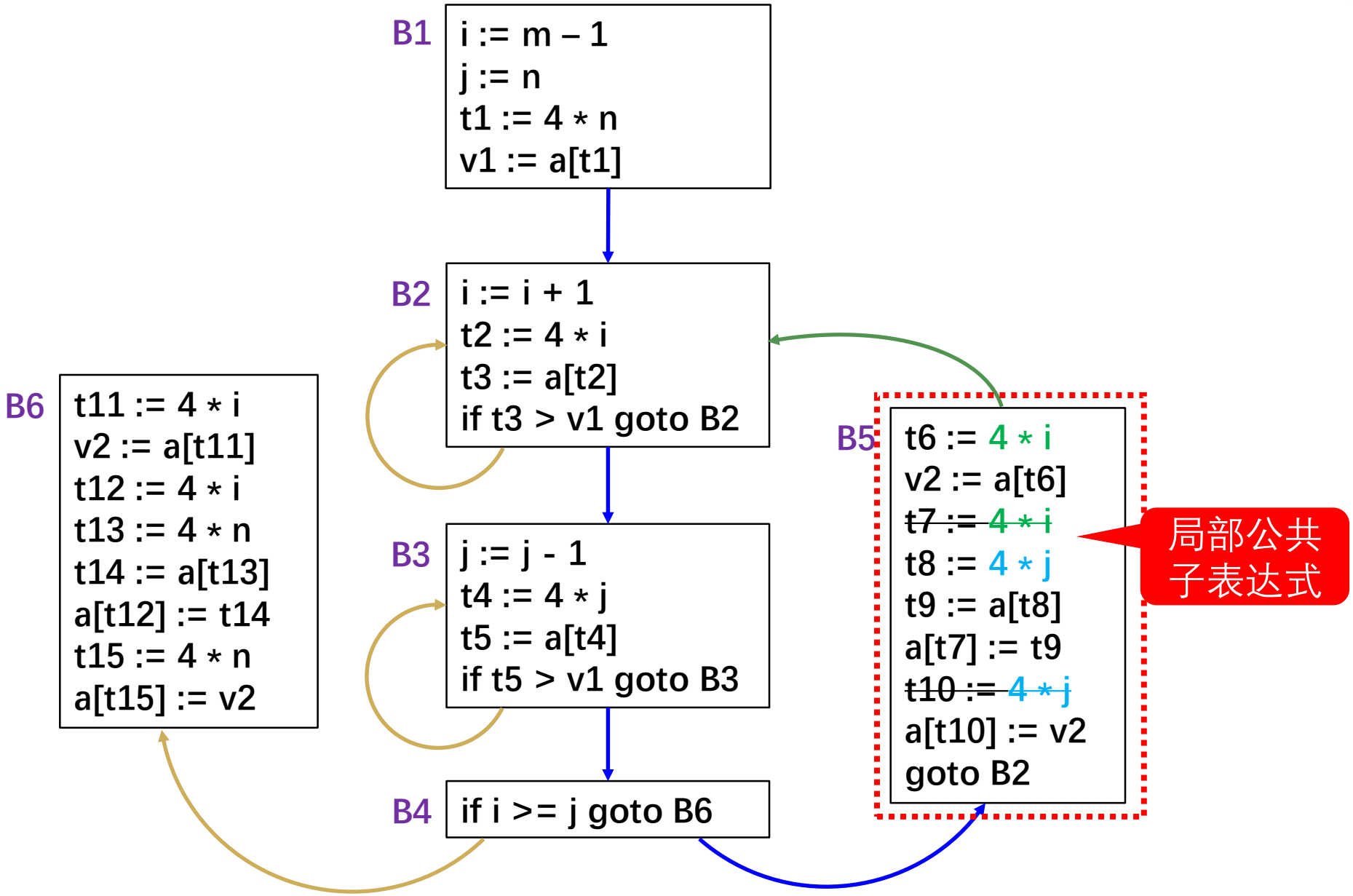
快排中的公共子表达式删除



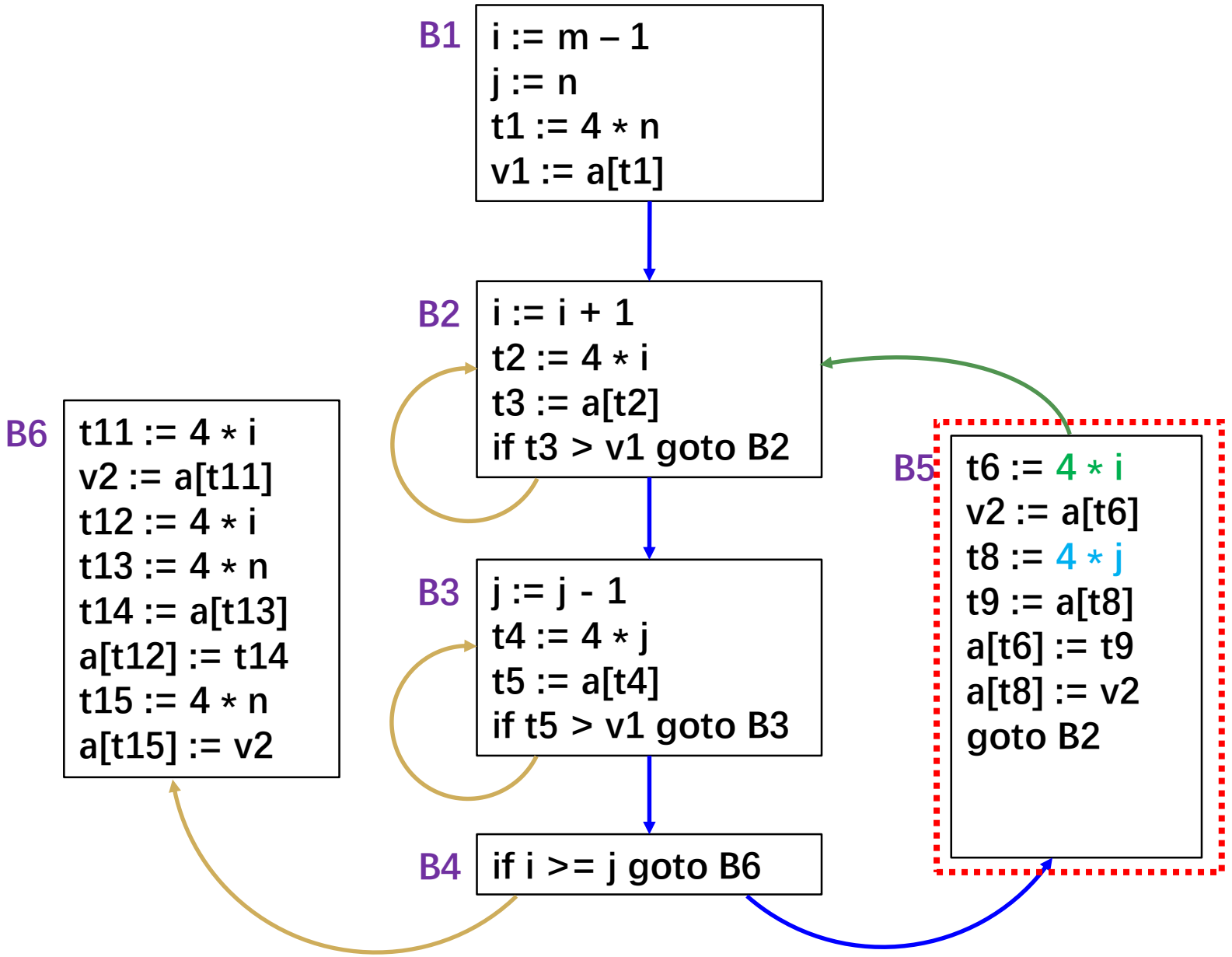
快排中的公共子表达式删除



快排中的公共子表达式删除



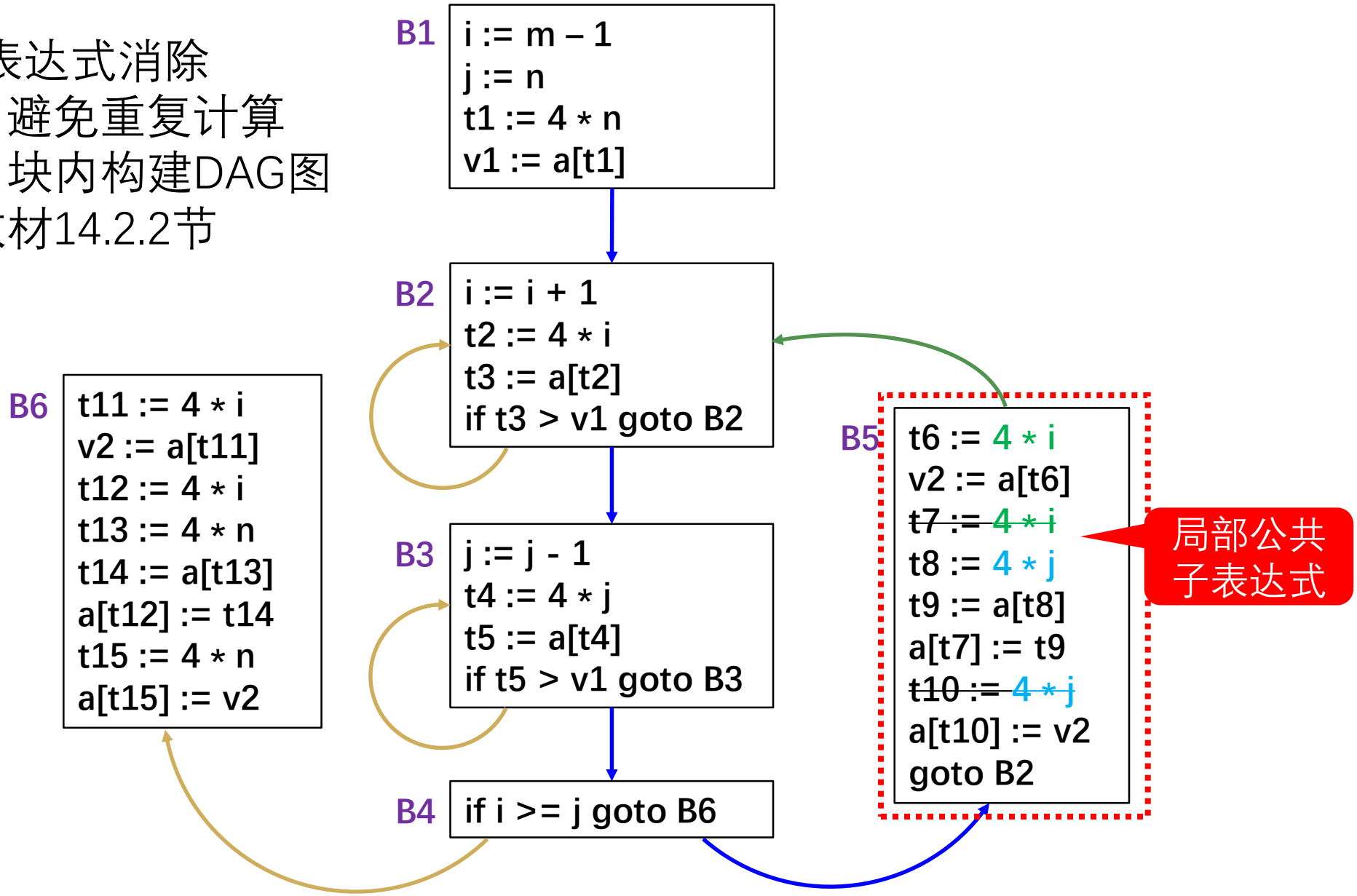
快排中的公共子表达式删除



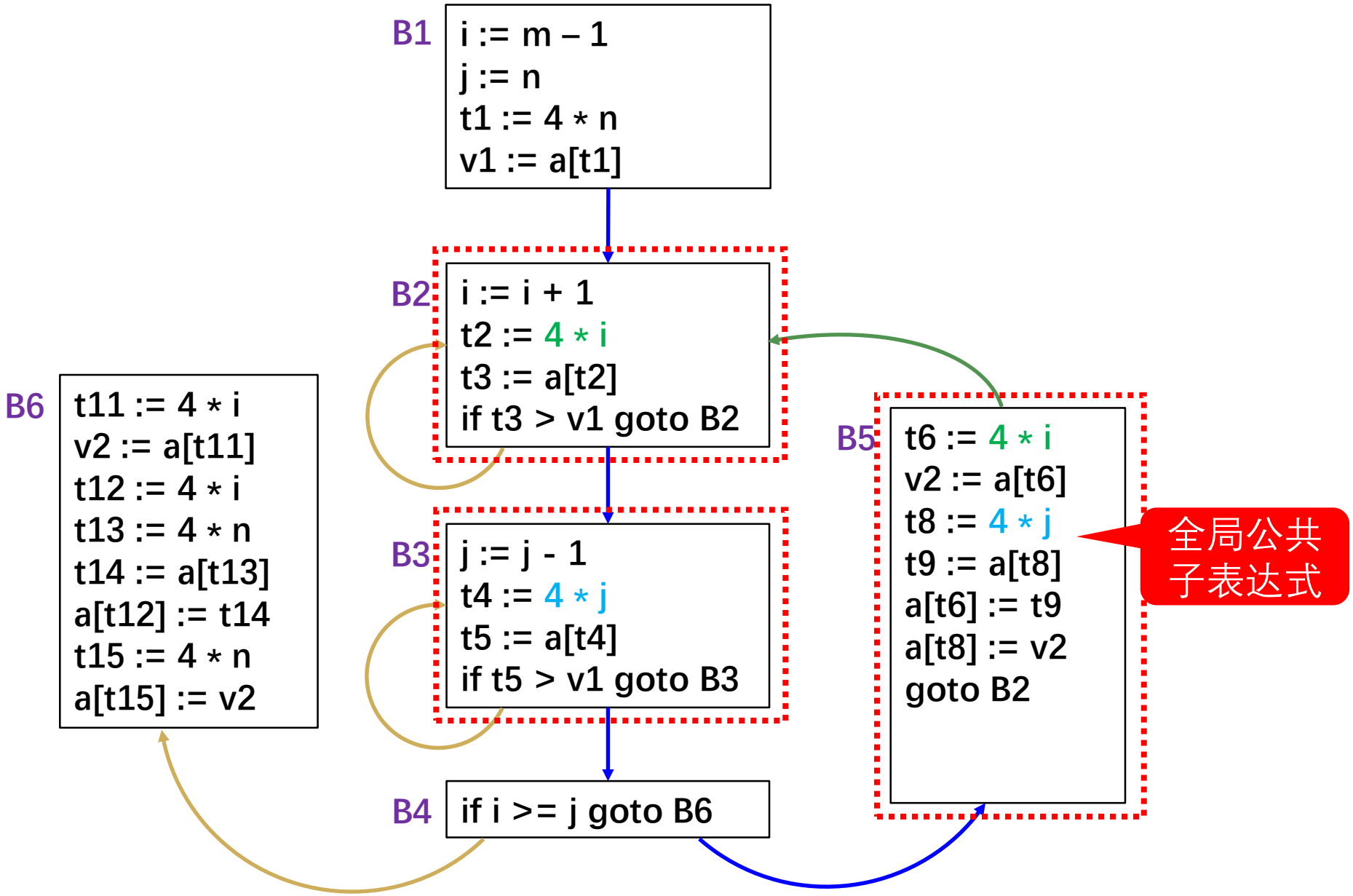
快排中的公共子表达式删除

局部公共子表达式消除

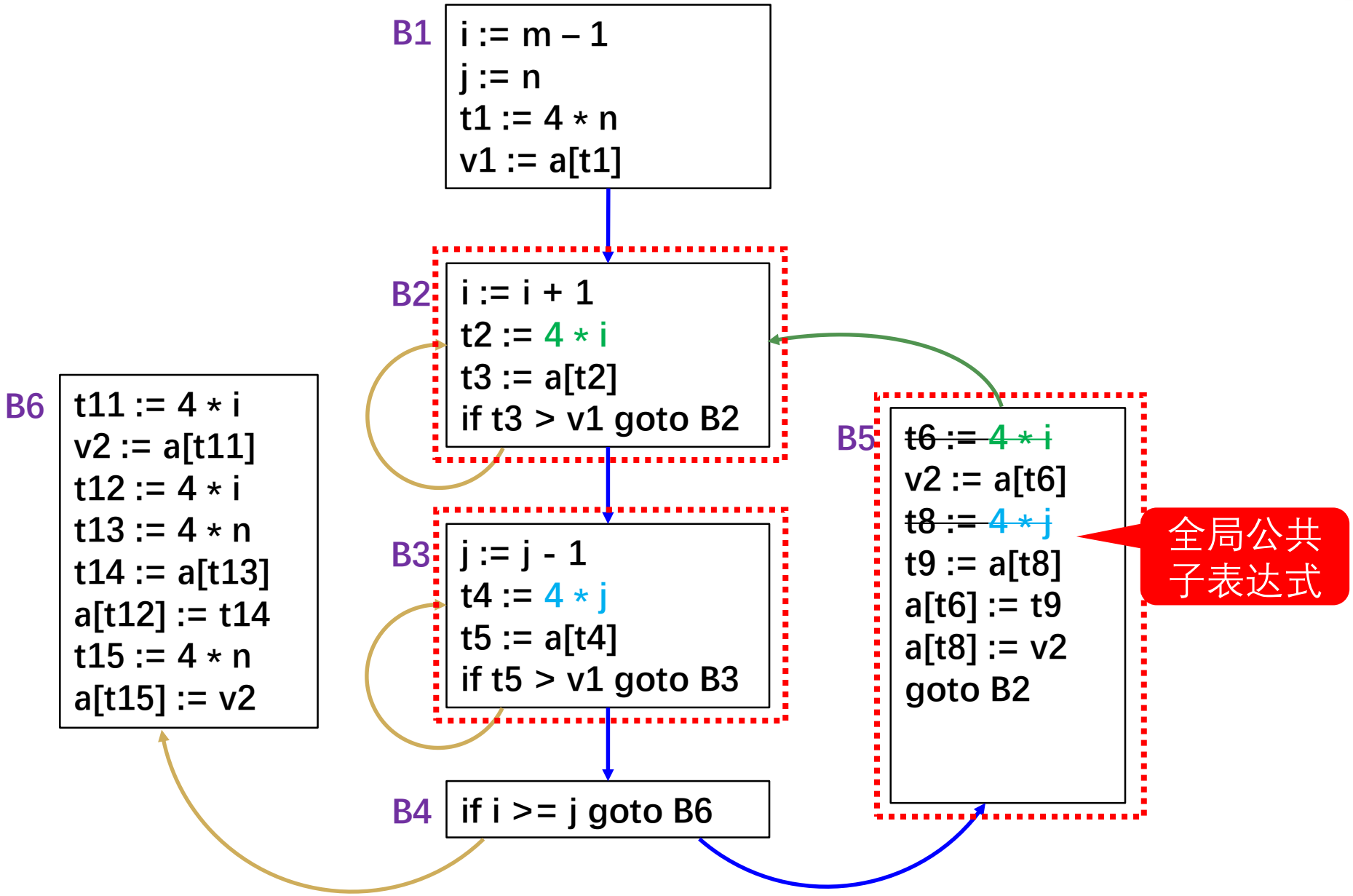
- 思想：避免重复计算
- 方法：块内构建DAG图
- 参考教材14.2.2节



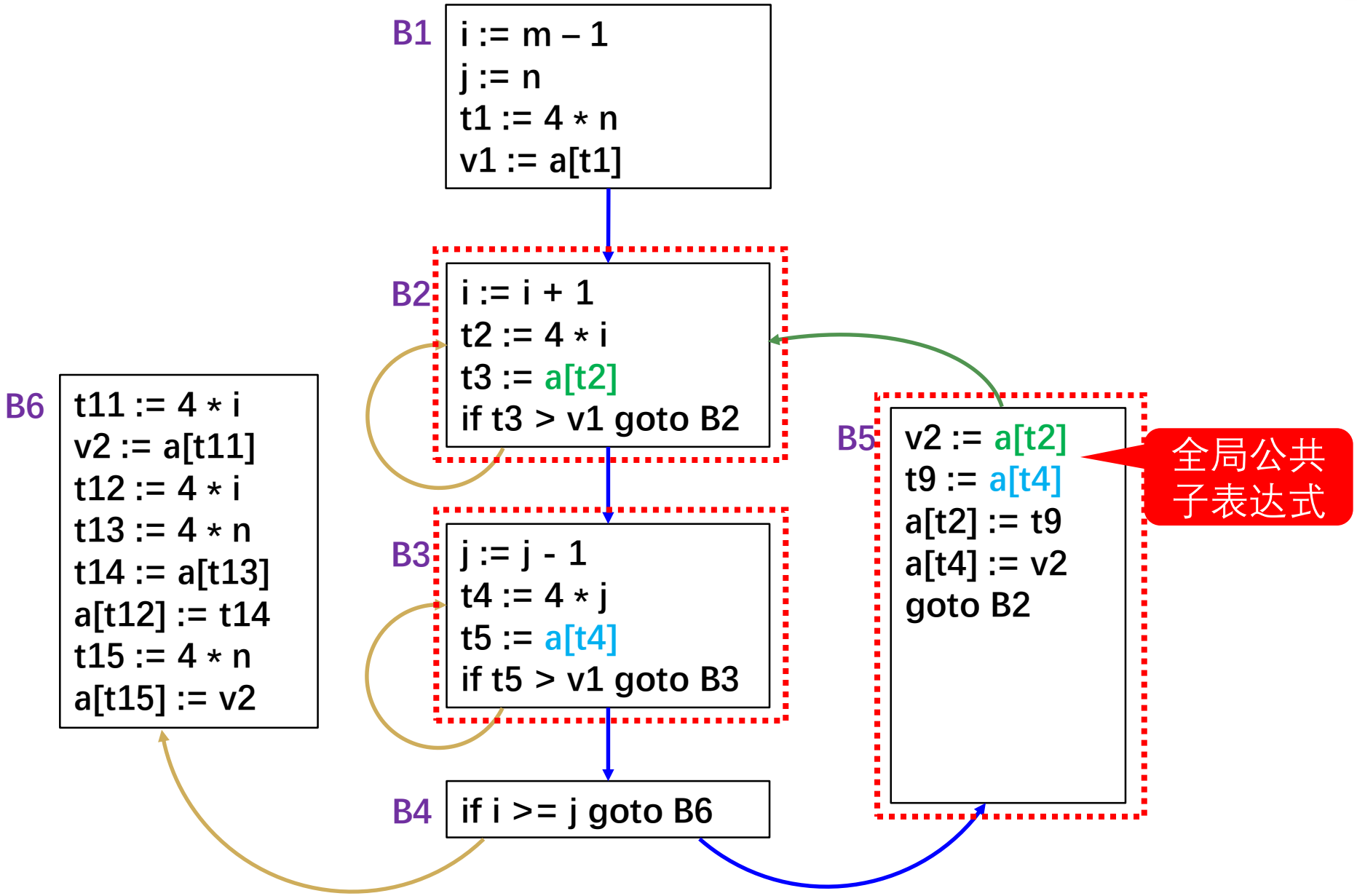
快排中的公共子表达式删除



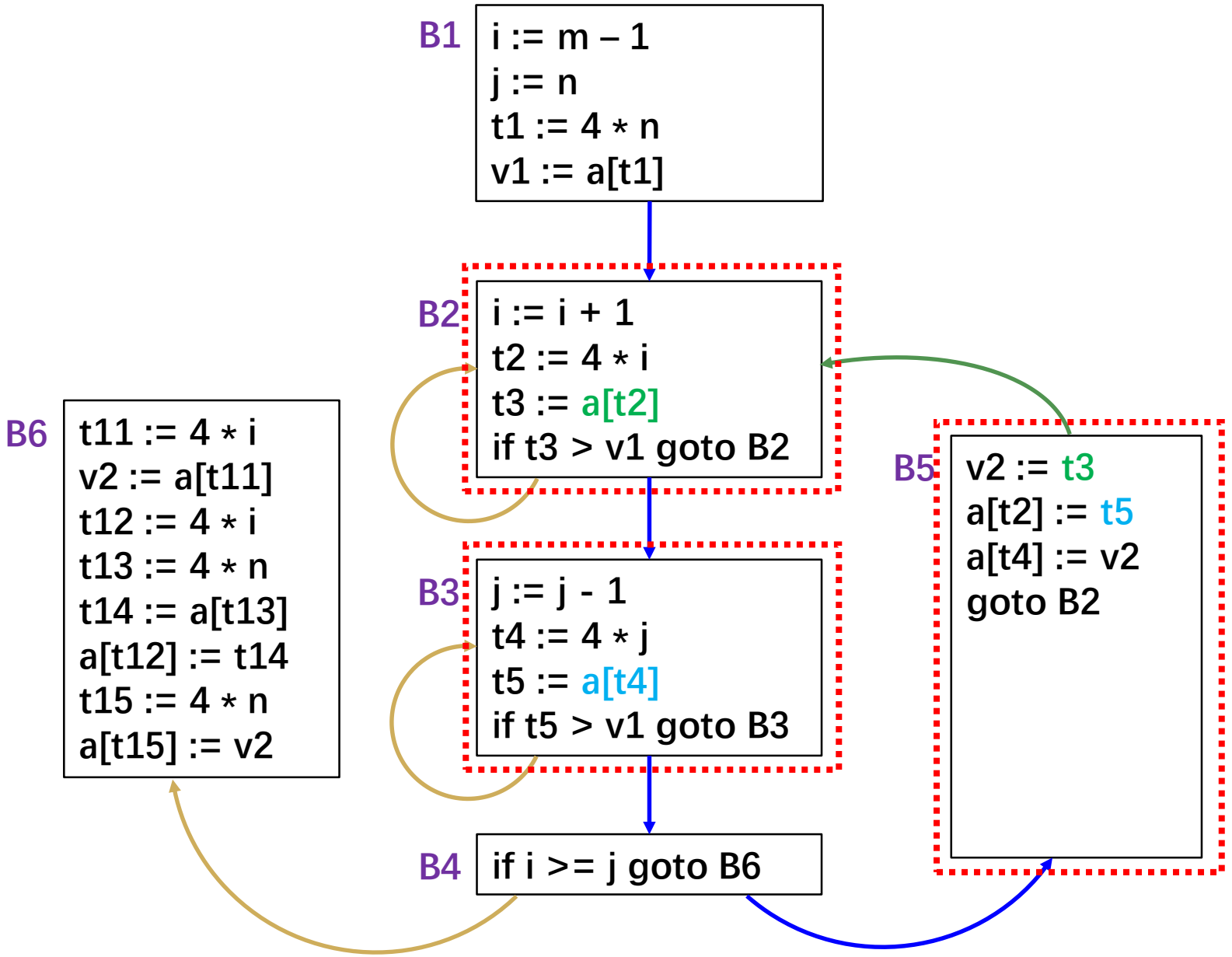
快排中的公共子表达式删除



快排中的公共子表达式删除



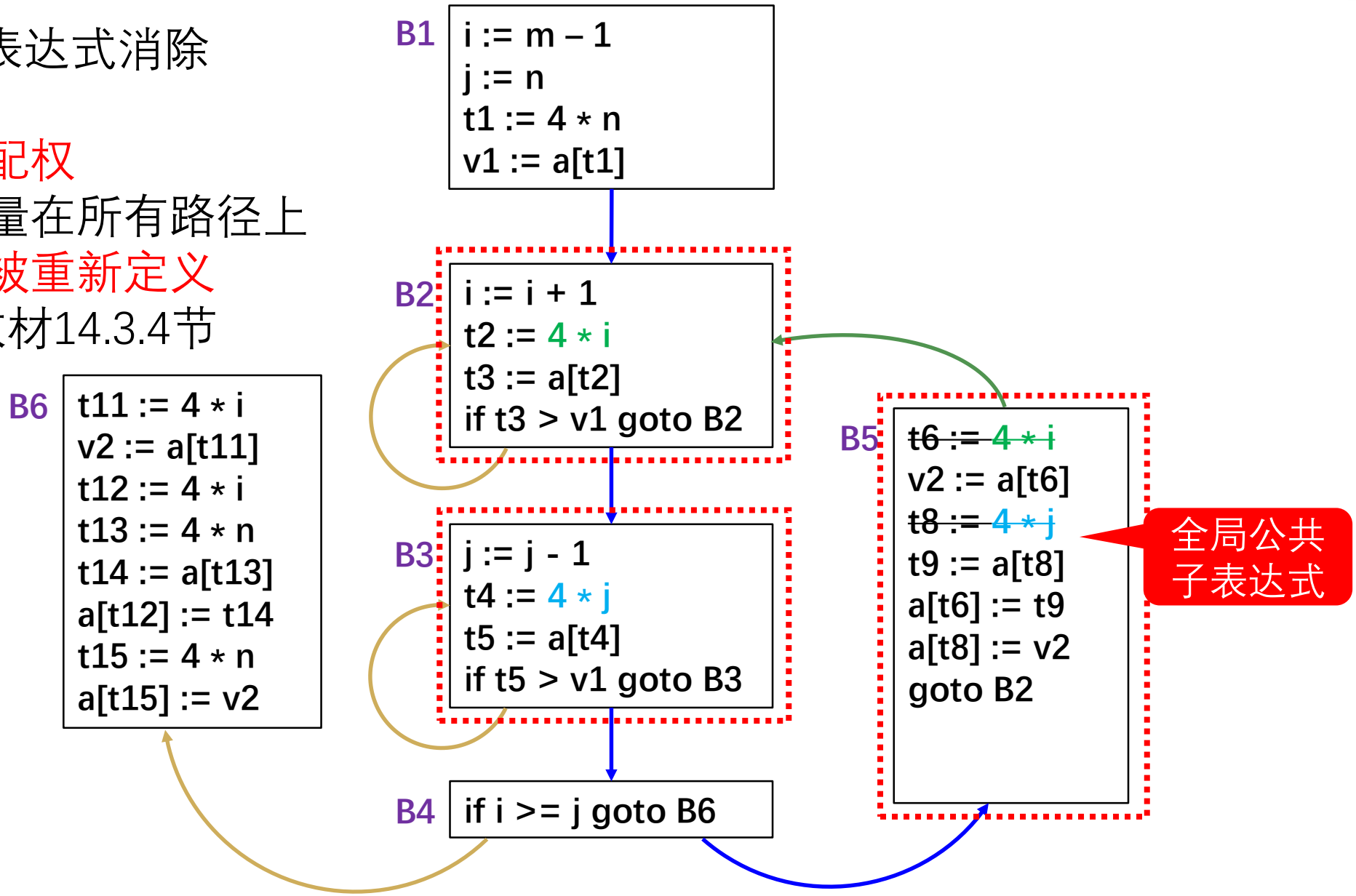
快排中的公共子表达式删除



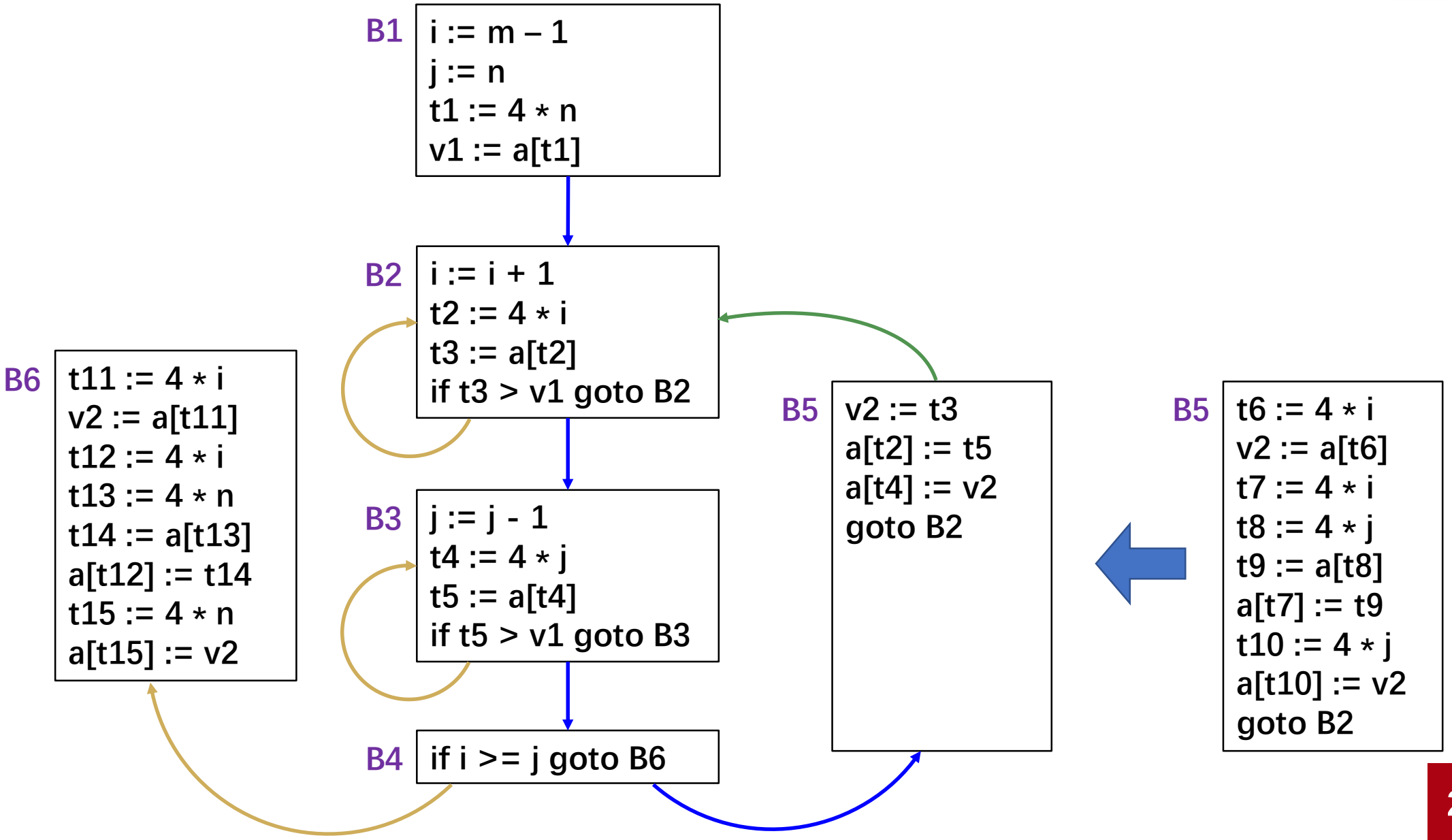
快排中的公共子表达式删除

全局公共子表达式消除

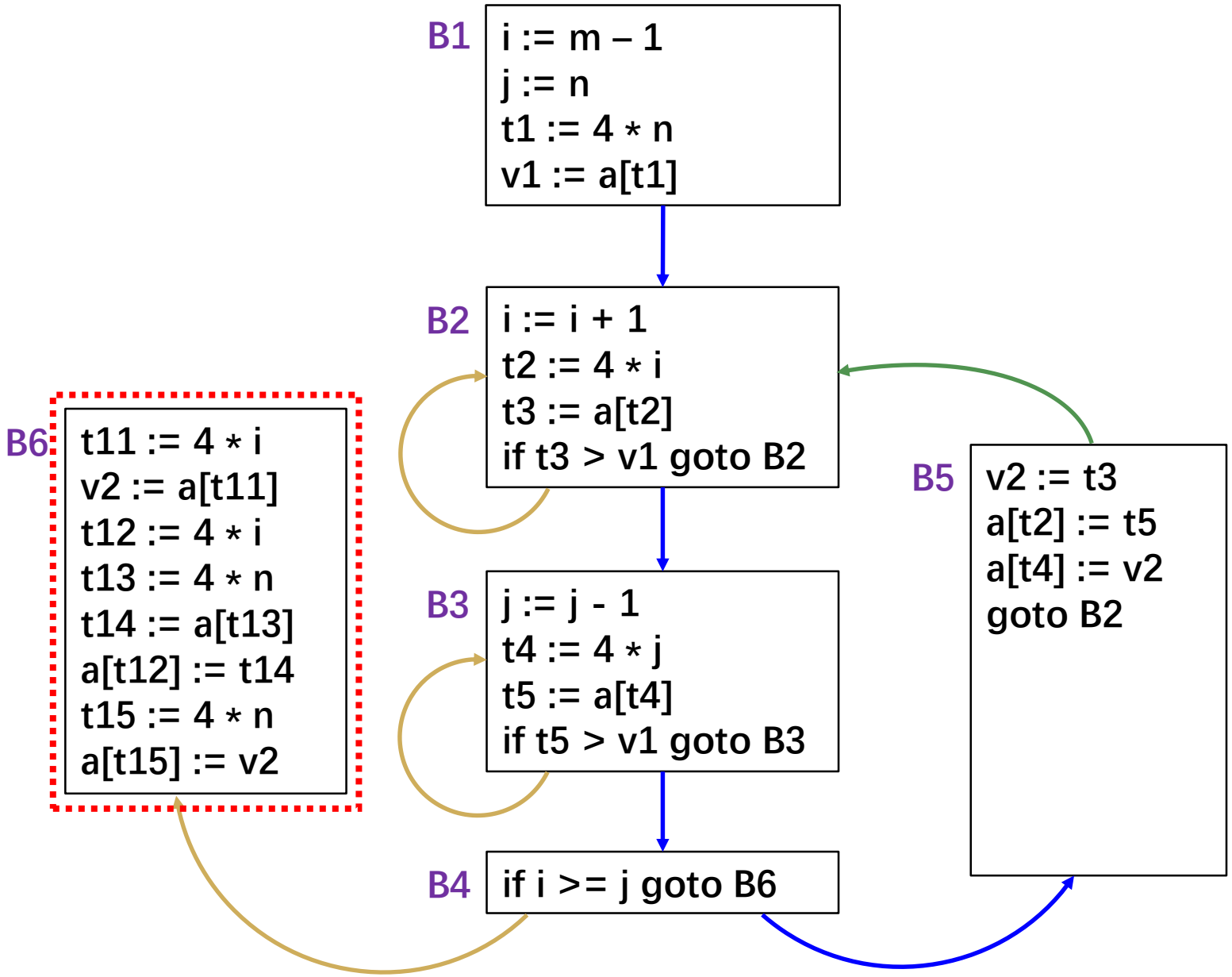
- 算法：
  - 支配权
  - 变量在所有路径上未被重新定义
- 参考教材14.3.4节



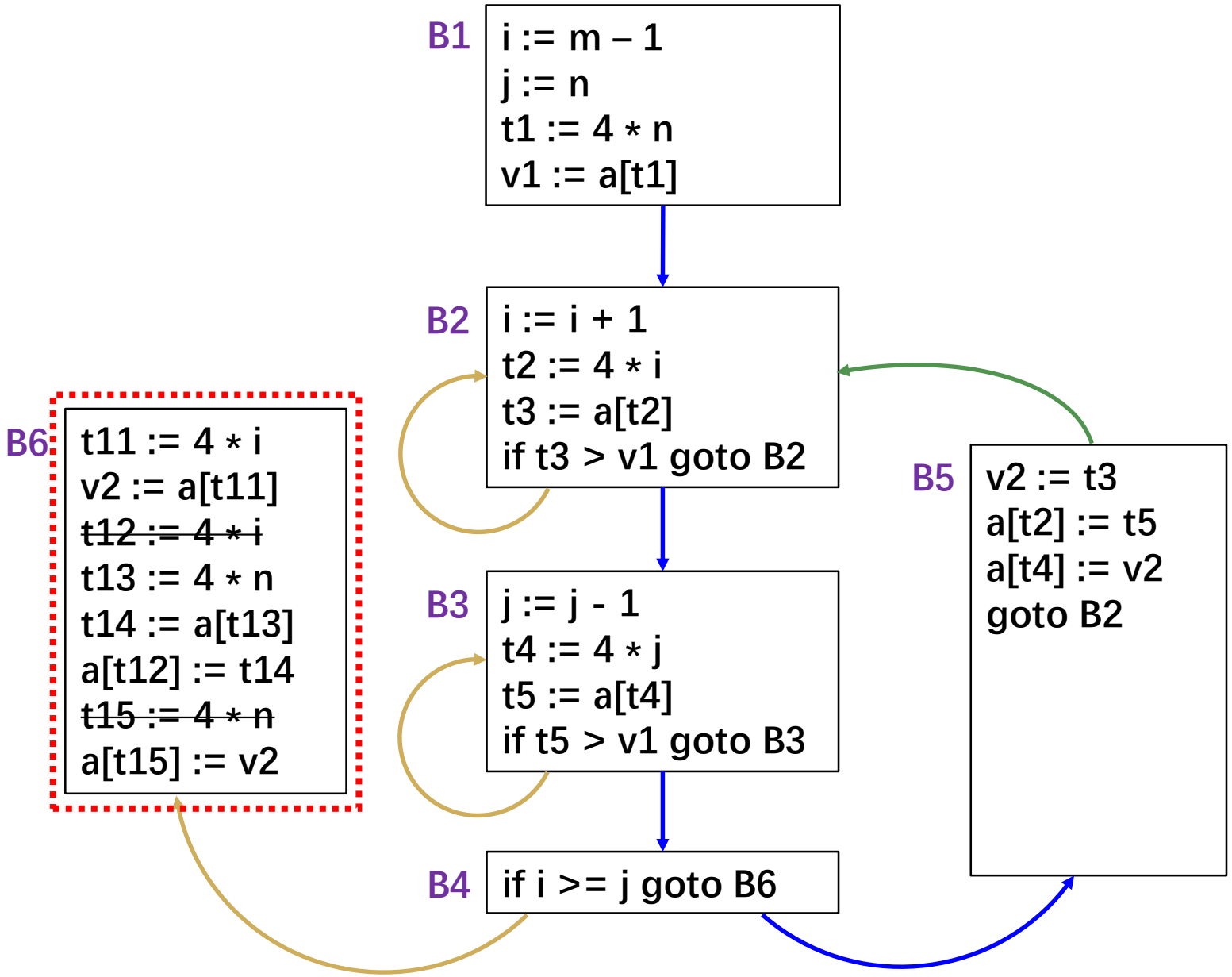
快排中的公共子表达式删除



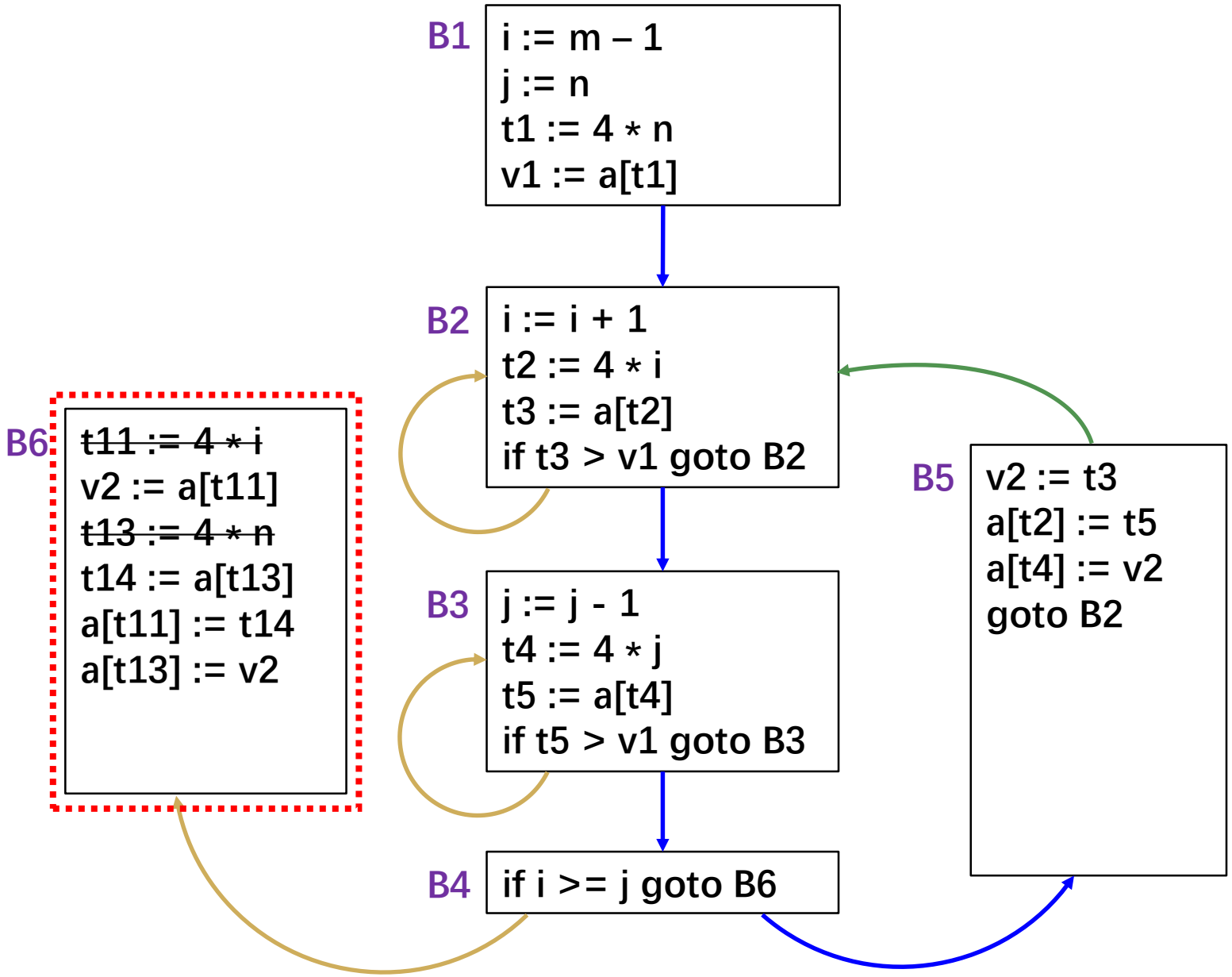
快排中的公共子表达式删除



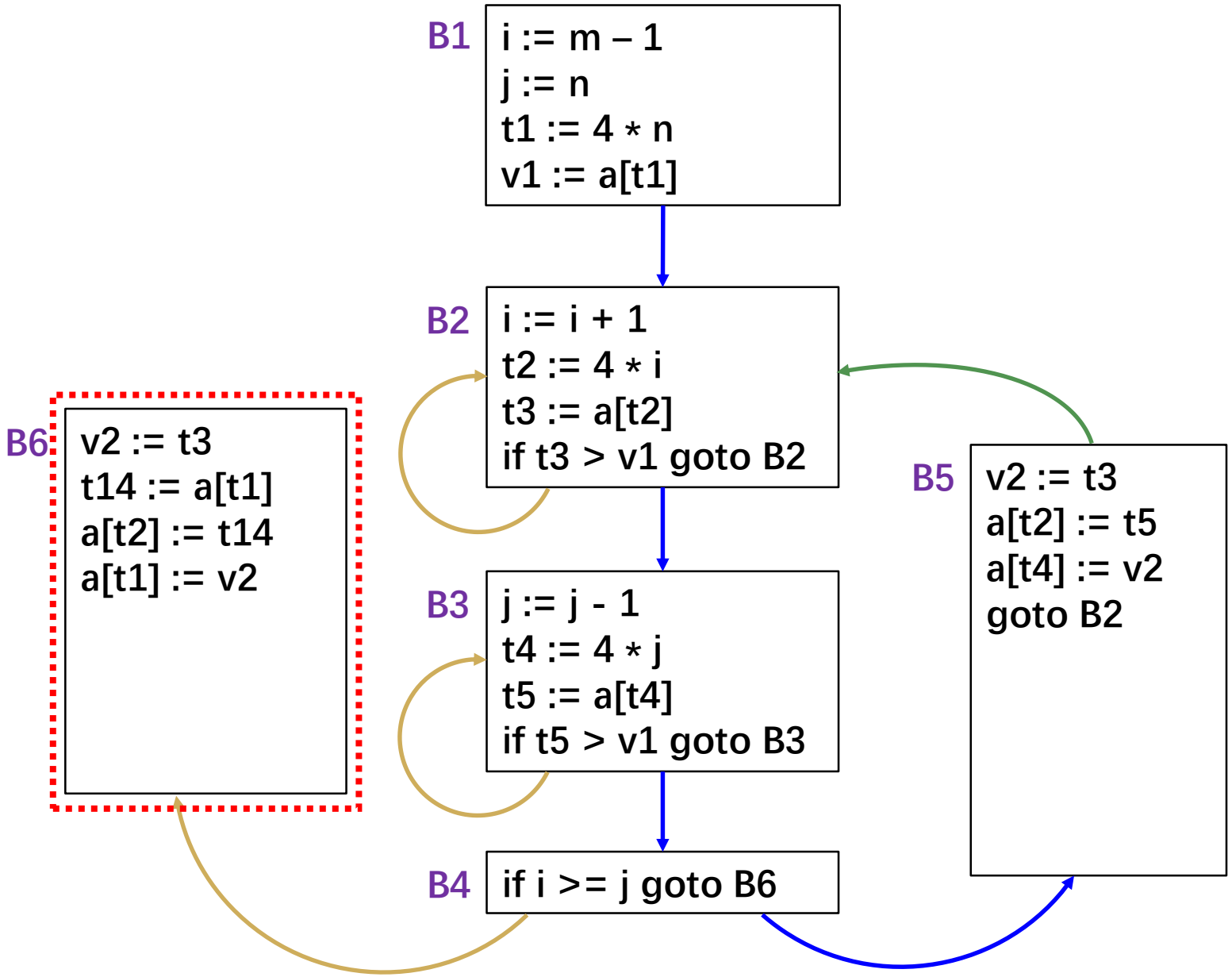
快排中的公共子表达式删除



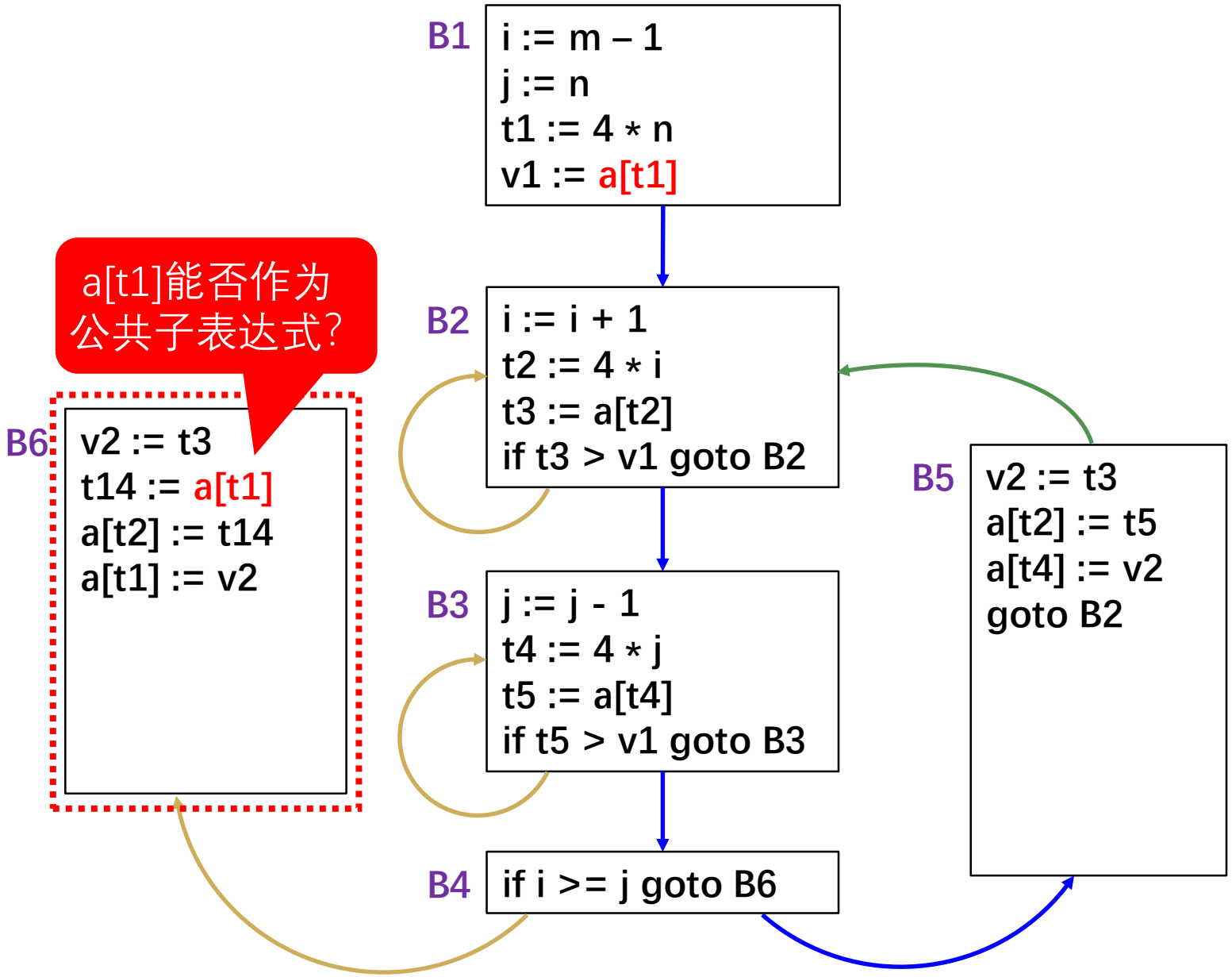
快排中的公共子表达式删除



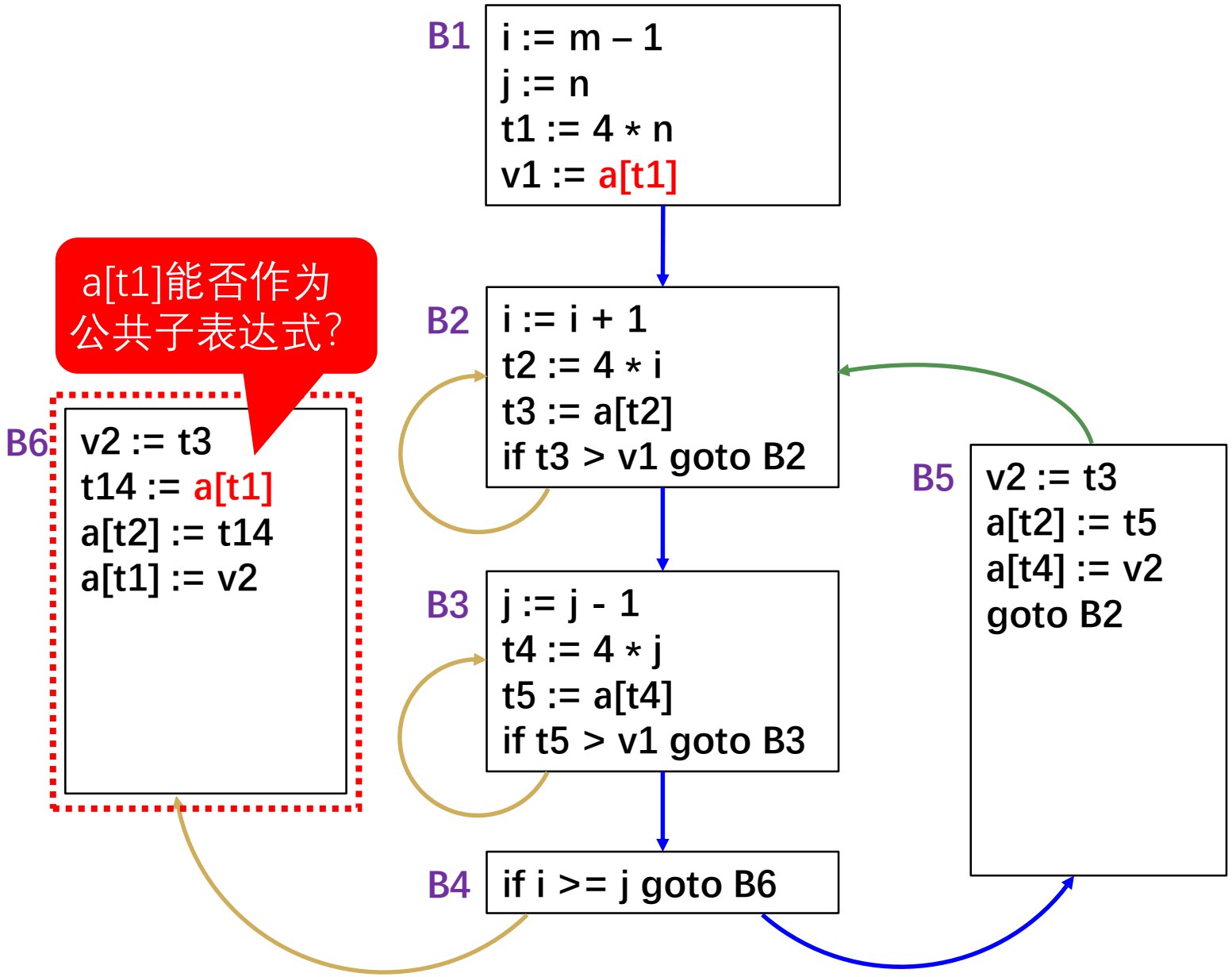
快排中的公共子表达式删除



快排中的公共子表达式删除



快排中的公共子表达式删除

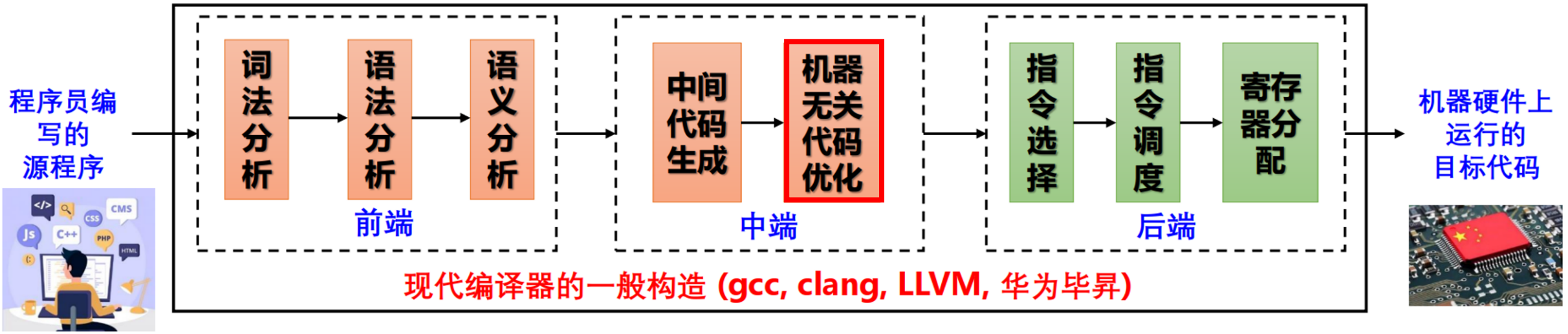


不稳妥。
因为B5有对a[t2]和a[t4]的赋值，改变了数组中的元素值。

理论联系实际——课外延伸阅读

- **工程实现公共子表达式删除的关键理论与技术点**
  - 可用数据流分析方法（教材14.3节）
  - 公共子表达式删除的工业界实现代码
    - 文档链接：https://llvm.org/doxygen/EarlyCSE_8cpp.html
    - 源码链接：https://llvm.org/doxygen/EarlyCSE_8cpp_source.html
  - 基于Global Value Numbering算法的工业界实现代码
    - 文档链接：https://llvm.org/docs/Passes.html#gvn-global-value-numbering
    - 源码链接：https://llvm.org/doxygen/GVN_8cpp_source.html

本节提纲



- 代码优化的定义及背景
- **常见的优化方法**
  - 公共子表达式删除
  - 死代码删除、复制传播、常量合并
  - 循环系列优化

死代码删除

- 死代码是指计算的结果永远不被引用的语句。

例：
为便于调试，可能在程序中加打印语句，测试通过后改成右边的形式

```
debug = true;                |    debug = false;
. . .                        |    . . .
if (debug) print …           |    if (debug) print …
```

死代码删除

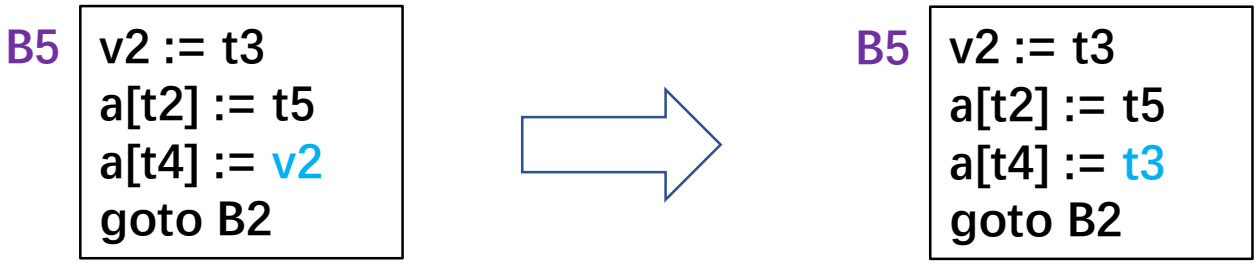
- 死代码是指计算的结果永远不被引用的语句
- 一些优化变换可能会引入死代码
  - 如：复制传播、常量合并

例：
为便于调试，可能在程序中加打印语句，测试通过后改成右边的形式

debug = true;	debug = false;
. . .	. . .
if (debug) print …	if (debug) print …

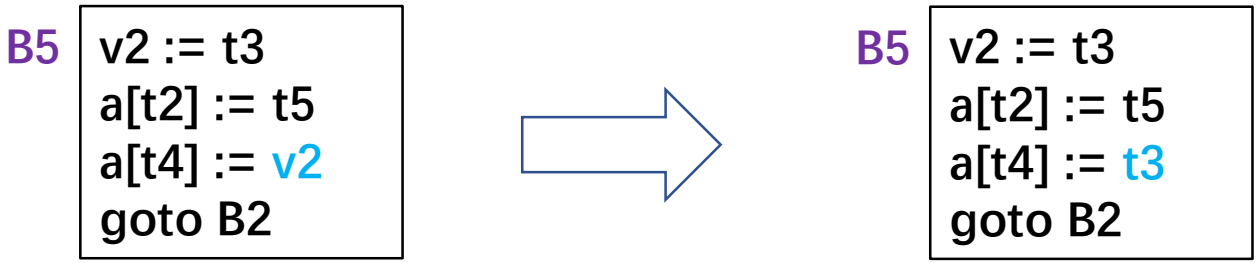
复制传播

- 定义：在复制语句x = y之后尽可能用y代替x



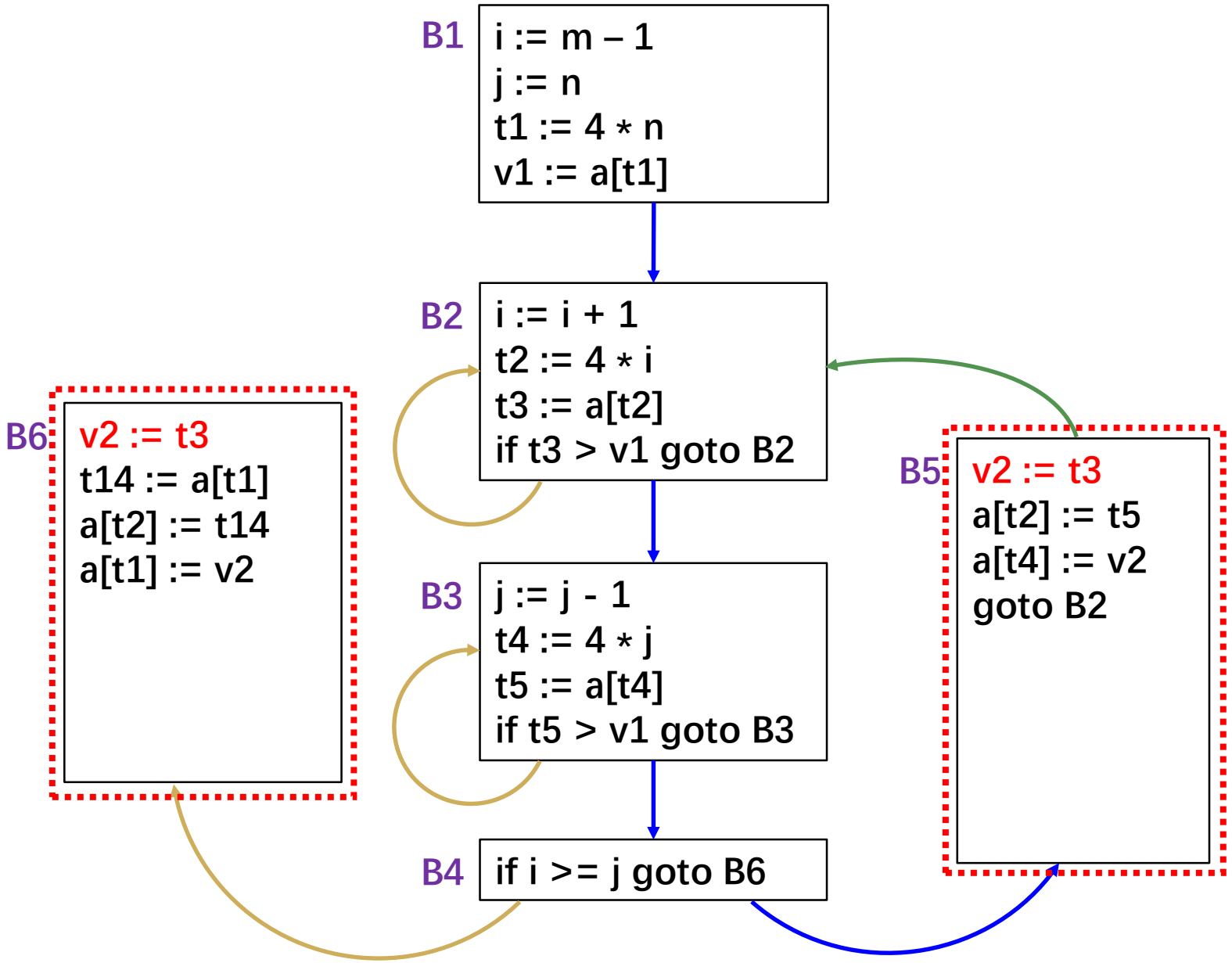
复制传播

- 定义：在复制语句x = y之后尽可能用y代替x

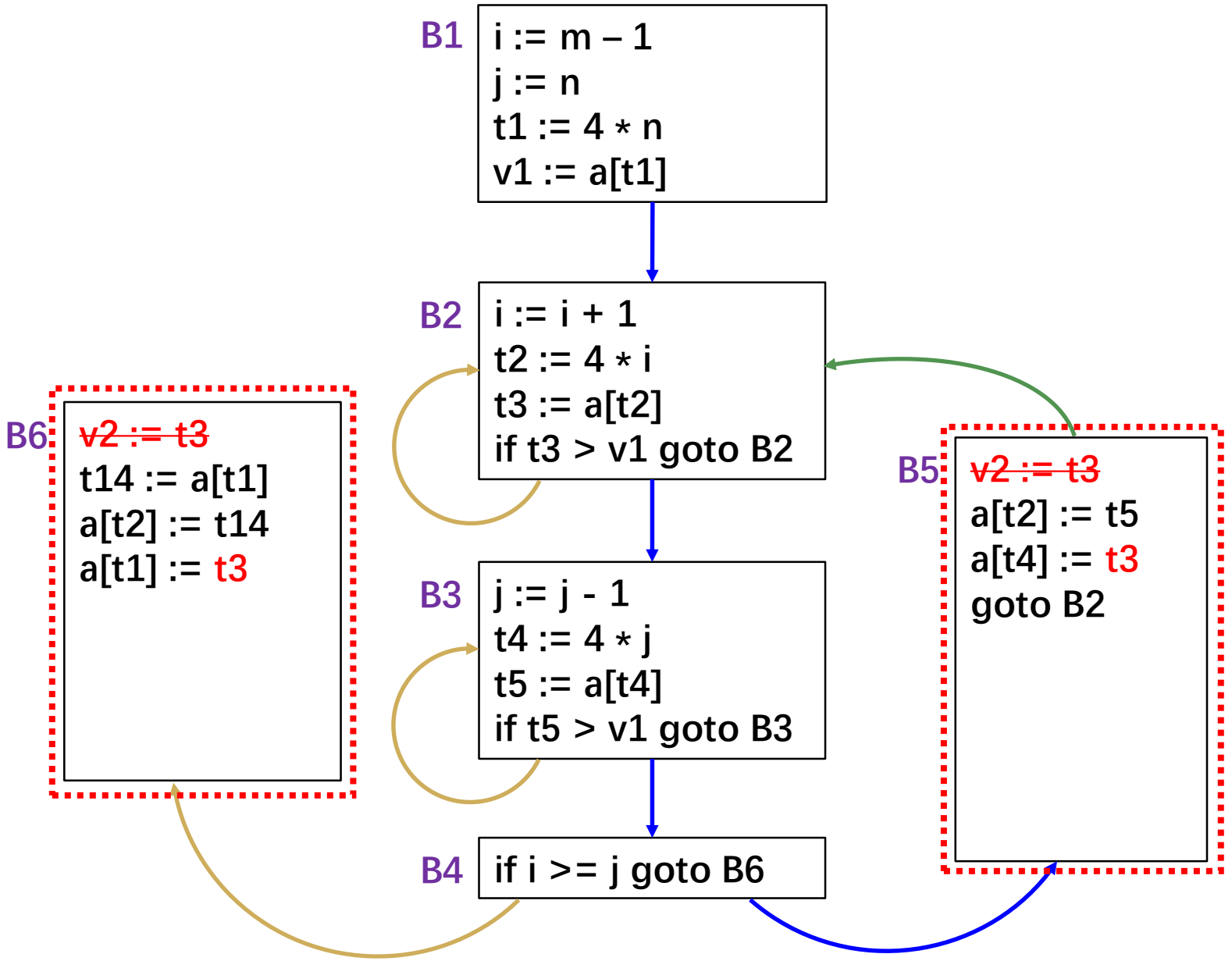


- 常用的公共子表达式删除和其他一些优化会引入一些复制语句
- 复制传播本身没有优化的意义，但可以给死代码删除创造机会

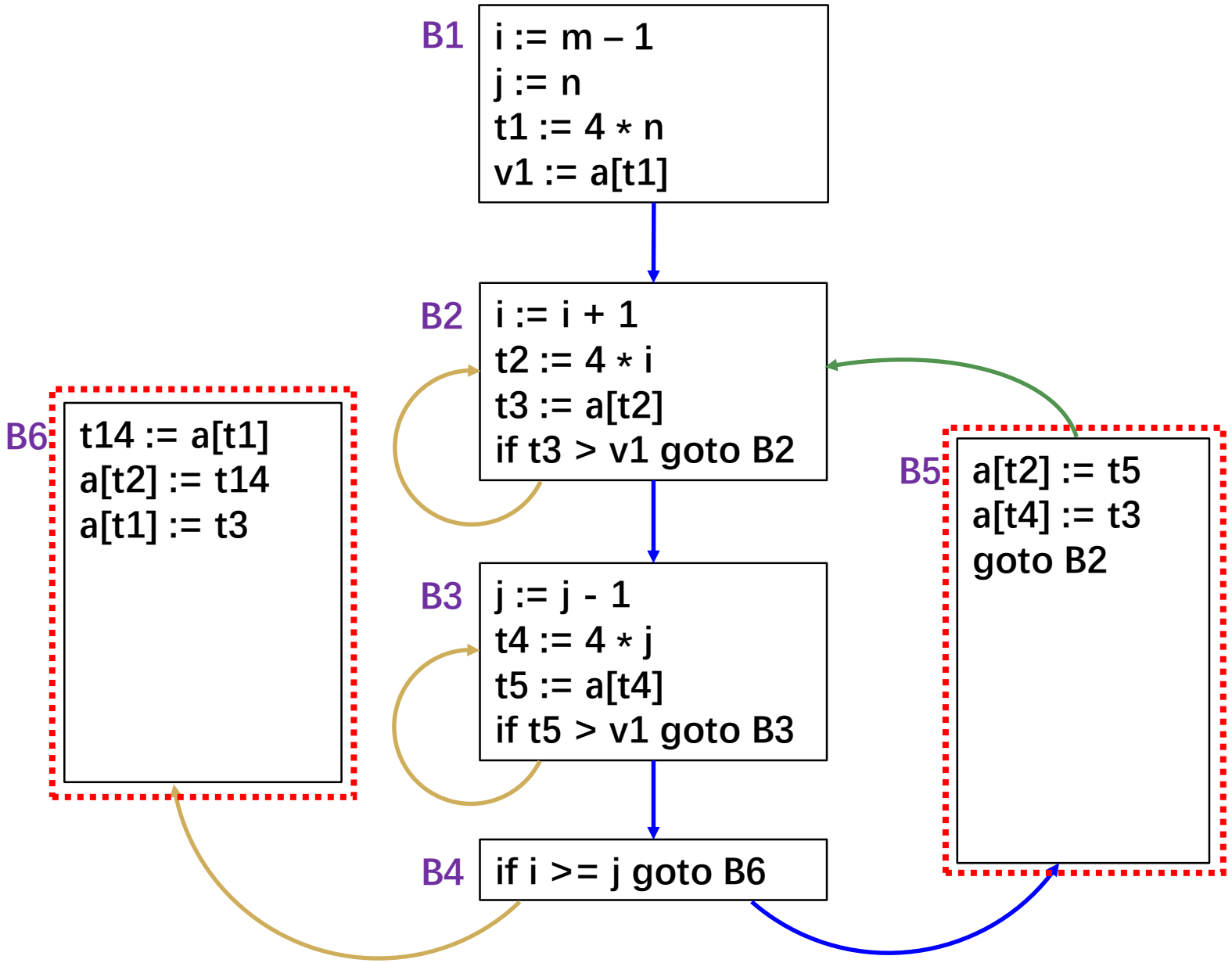
快排中的死代码删除


B1
i := m – 1
j := n
t1 := 4 * n
v1 := a[t1]
B2
i := i + 1
t2 := 4 * i
t3 := a[t2]
if t3 > v1 goto B2
B3
j := j - 1
t4 := 4 * j
t5 := a[t4]
if t5 > v1 goto B3
B4
if i >= j goto B6
B5
v2 := t3
a[t2] := t5
a[t4] := v2
goto B2
B6
v2 := t3
t14 := a[t1]
a[t2] := t14
a[t1] := v2

快排中的死代码删除

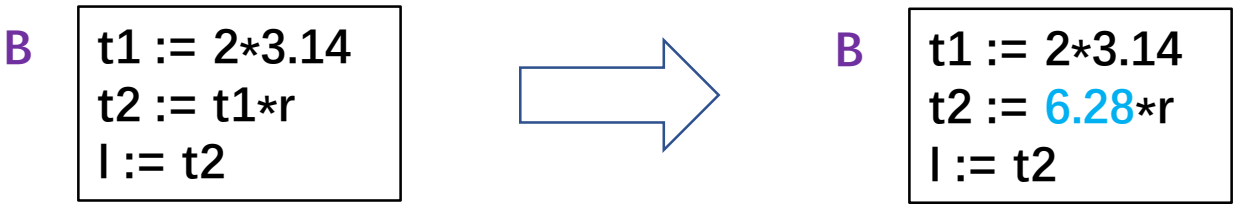

B1
i := m – 1
j := n
t1 := 4 * n
v1 := a[t1]
B2
i := i + 1
t2 := 4 * i
t3 := a[t2]
if t3 > v1 goto B2
B3
j := j - 1
t4 := 4 * j
t5 := a[t4]
if t5 > v1 goto B3
B4
if i >= j goto B6
B5
v2 := t3
a[t2] := t5
a[t4] := t3
goto B2
B6
v2 := t3
t14 := a[t1]
a[t2] := t14
a[t1] := t3

快排中的死代码删除


B1
i := m – 1
j := n
t1 := 4 * n
v1 := a[t1]
B2
i := i + 1
t2 := 4 * i
t3 := a[t2]
if t3 > v1 goto B2
B3
j := j - 1
t4 := 4 * j
t5 := a[t4]
if t5 > v1 goto B3
B4
if i >= j goto B6
B5
a[t2] := t5
a[t4] := t3
goto B2
B6
t14 := a[t1]
a[t2] := t14
a[t1] := t3

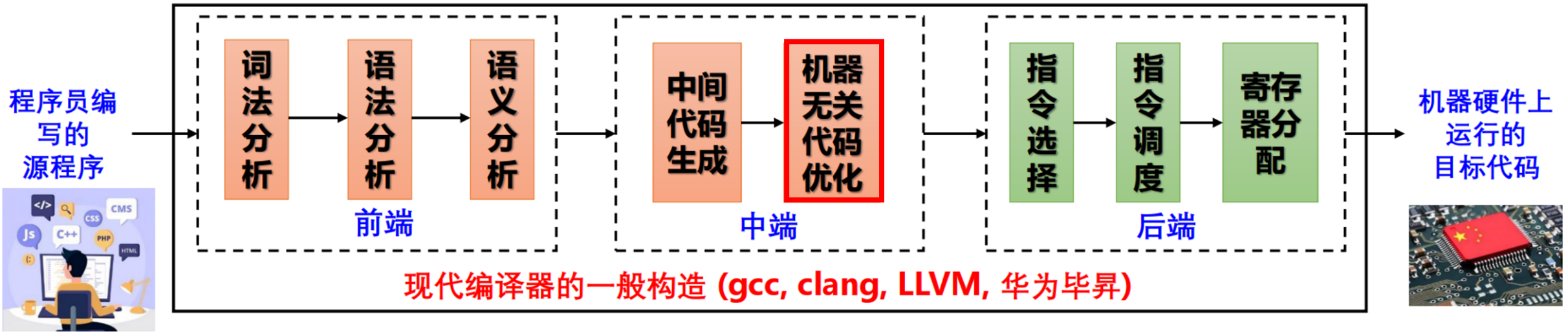
常量合并

- 如果在编译时刻推导出一个表达式的值是常量，就可以使用该常量来代替这个表达式。
  - 例：计算圆周长的表达式 $l = 2*3.14*r$



- 常量合并本身没有优化意义，但可以给死代码删除创造机会。

本节提纲



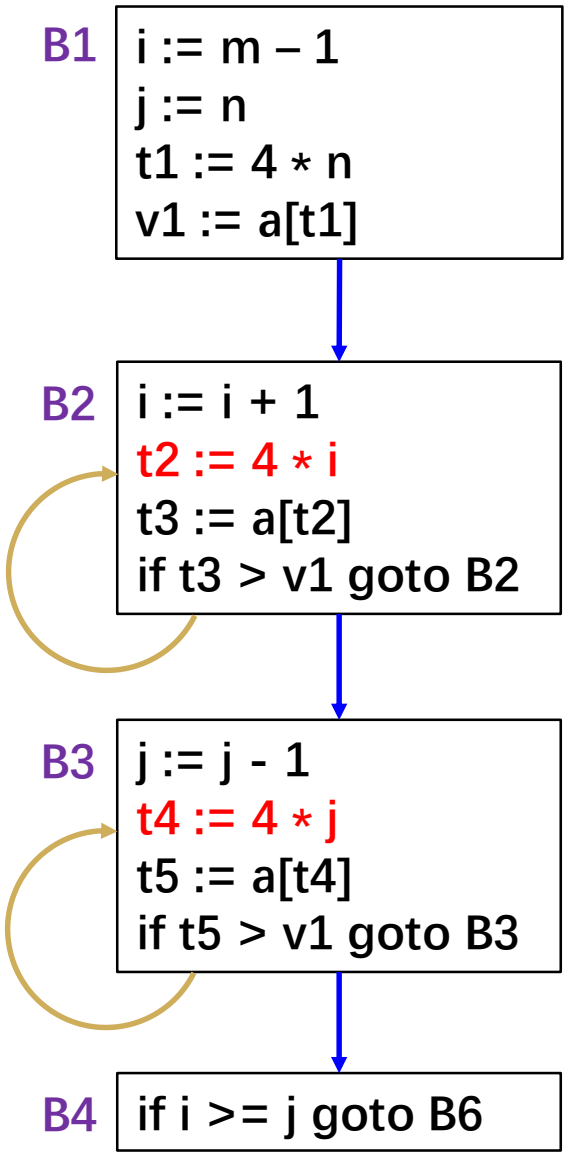
- 代码优化的定义及背景
- 常见的优化方法
  - 公共子表达式删除
  - 死代码删除、复制传播、常量合并
  - 循环系列优化

循环中的强度削弱

- **在循环中的代码会被执行多次。**
  - 迭代次数越多，执行时间越长
  - 降低每次迭代计算的复杂度是循环优化的重要方法
- **潜在的优化可能——强度削弱（Strength Reduction）**
  - 将程序中执行时间较长的运算替换为执行时间较短的运算

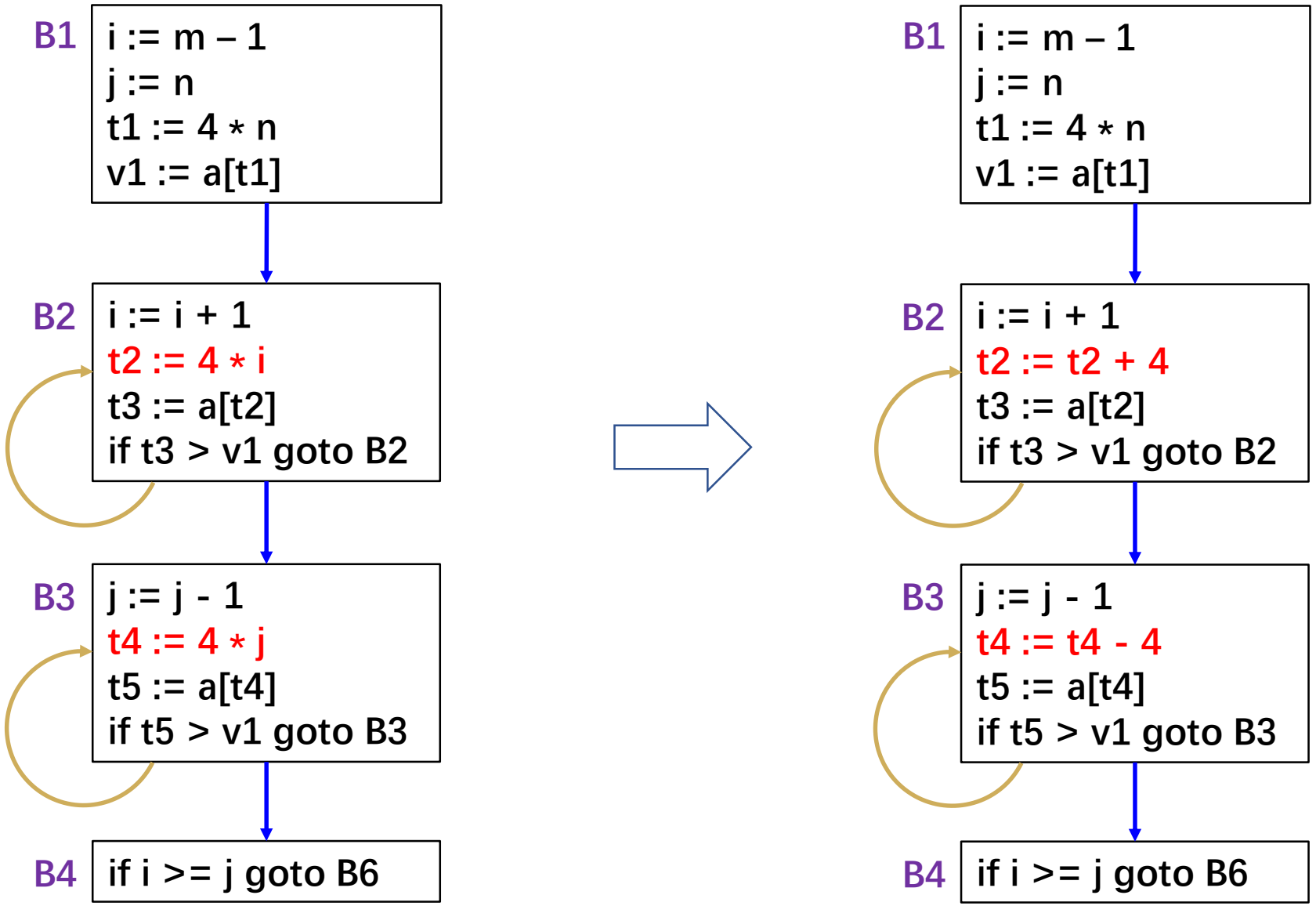
$2 * x$ 或者 $2.0 * x$	$x + x$
$x/2$	$x * 0.5$
x^2	$x * x$
$a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x^1 + a_0$	$((\ldots (a_n x + a_{n-1})x + a_{n-2})\ldots)x + a_1) + a_0$
开销较高的运算	开销较低的等价运算

循环中的强度削弱

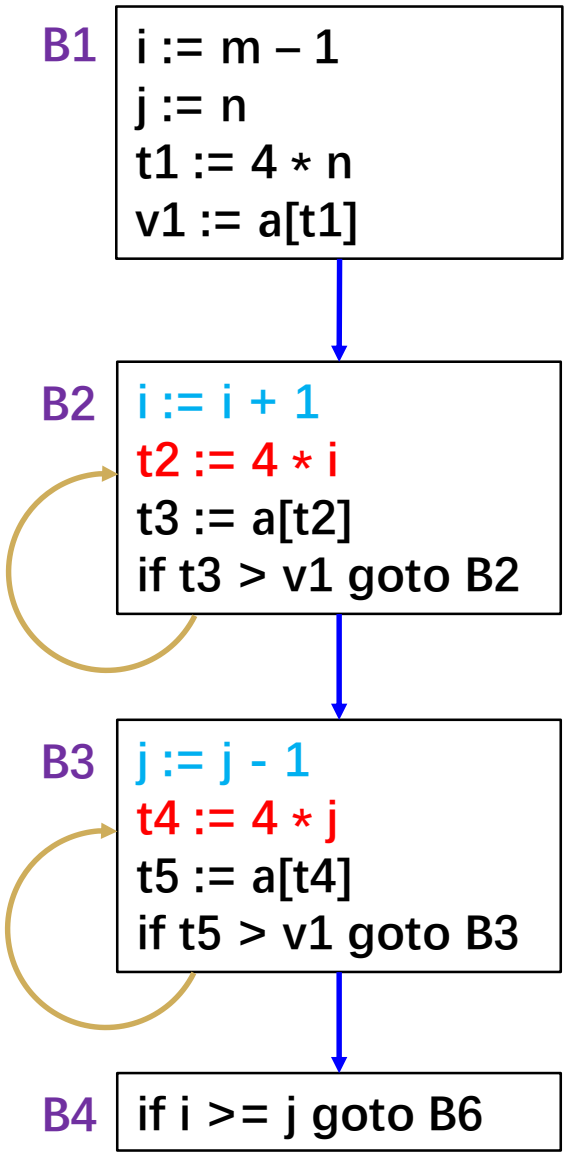


- **观察B2和B3中的变量t2和t4**
  - 步幅规律递增
- **归纳变量**
  - 如果存在一个常量c，使x的每一次赋值总是增加c，则称x为归纳变量。
  - 基本形式：x = x + c
  - 高级形式：x = c*i + d （c,d常量，i归纳变量）
  - 强度削弱：用增量运算（加或减）替代

循环中的强度削弱

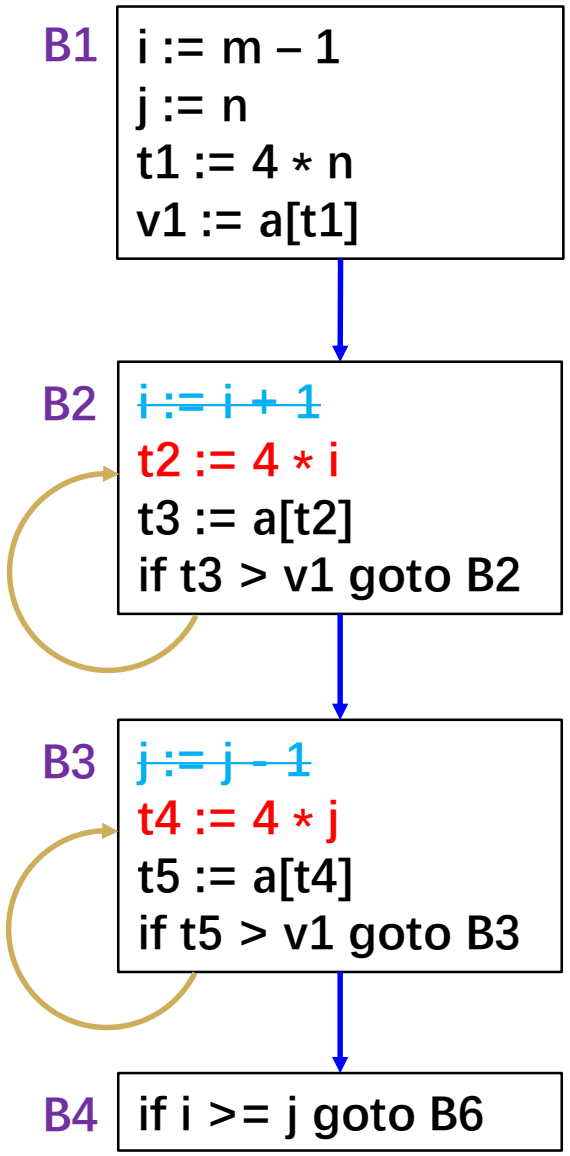

B1
i := m – 1
j := n
t1 := 4 * n
v1 := a[t1]
B2
i := i + 1
t2 := 4 * i
t3 := a[t2]
if t3 > v1 goto B2
B3
j := j - 1
t4 := 4 * j
t5 := a[t4]
if t5 > v1 goto B3
B4
if i >= j goto B6
B1
i := m – 1
j := n
t1 := 4 * n
v1 := a[t1]
B2
i := i + 1
t2 := t2 + 4
t3 := a[t2]
if t3 > v1 goto B2
B3
j := j - 1
t4 := t4 - 4
t5 := a[t4]
if t5 > v1 goto B3
B4
if i >= j goto B6

循环中的归纳变量删除



- i,j,t2和t4均为归纳变量
- 按照变化步调对归纳变量进行分组
  - i与t2
  - j与t4
- 删除冗余的归纳变量
  - 一个循环中，如一组归纳变量的值的变化保持步调一致，可只保留一个。

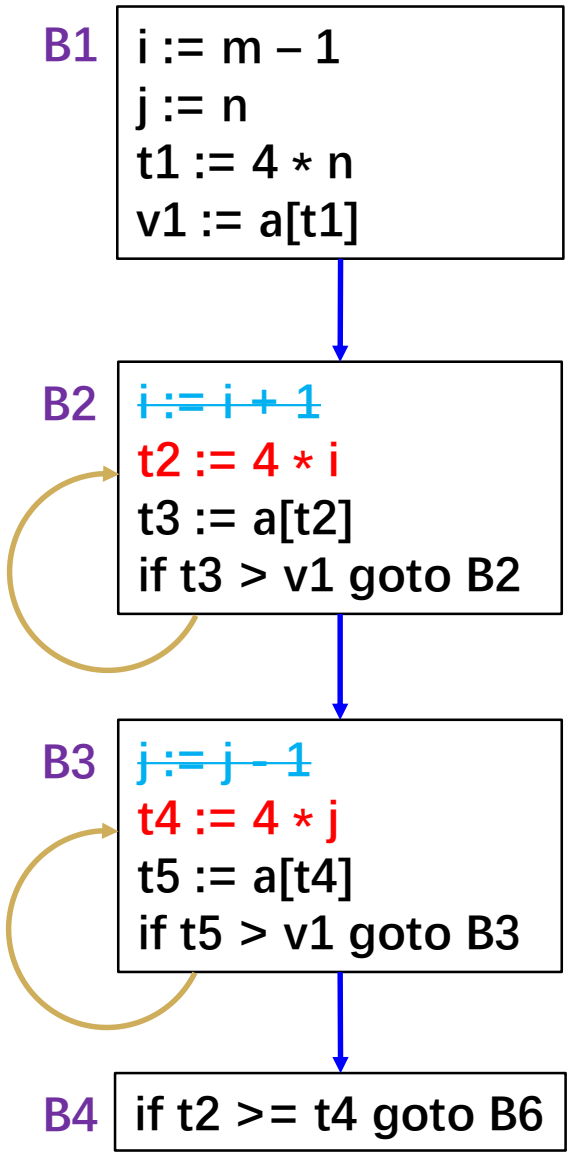
循环中的归纳变量删除



- i,j,t2和t4均为归纳变量
- 按照变化步调对归纳变量进行分组
  - i与t2
  - j与t4
- 删除冗余的归纳变量
  - 一个循环中，如一组归纳变量的值的变化保持步调一致，可只保留一个。

第一步：删除循环中对i和j的计算

循环中的归纳变量删除



- i,j,t2和t4均为归纳变量
- 按照变化步调对归纳变量进行分组
  - i与t2
  - j与t4
- 删除冗余的归纳变量
  - 一个循环中，如一组归纳变量的值的变化保持步调一致，可只保留一个。

第一步：删除循环中对i和j的计算
第二步：将对i和j的引用分别替换为t2和t4

理论联系实际——课外延伸阅读

- **工程实现循环优化的关键理论与技术点**
  - 如何识别归纳变量
    - 文档链接：https://llvm.org/docs/Passes.html#iv-users-induction-variable-users
    - 源码链接：https://llvm.org/doxygen/IVUsers_8cpp_source.html
  - 强度削弱的工业界实现代码
    - 文档链接：https://llvm.org/docs/Passes.html#loop-reduce-loop-strength-reduction
    - 源码链接：https://llvm.org/doxygen/LoopStrengthReduce_8cpp.html
  - 归纳变量删除的工业界实现代码
    - 文档链接：https://llvm.org/docs/Passes.html#indvars-canonicalize-induction-variables
    - 源码链接：https://llvm.org/doxygen/IndVarSimplify_8cpp.html

结束语

- **代码优化是编译技术的重要组成部分，是发挥硬件能力、提升编程水平的重要技术手段。**

- **常见的代码优化方法有循环优化和公共子表达式删除等**
  - 循环的强度削弱和归纳变量删除依赖于归纳变量识别
  - 公共子表达式删除依赖于公共子表达式的识别
  - 可以看出，大部分的高级优化均需要利用代码分析

图灵奖获得者——法兰·艾伦

- 由于在编译优化方面的杰出贡献被授予2006年计算机图灵奖
- 是世界上第一位获得图灵奖的女科学家
- 重要的理论与实践工作有：
  - Program Optimization, 1966
  - Control Flow Analysis, 1970
  - A Basis for Program Optimization, 1970
  - A Catalog of Optimizing Transformations, 1971
  - ……



Frances Allen
(1932 - 2020)
ACM/IEEE Fellow
美国科学院院士